Accelerating K-Means Clustering with Grid-Based Partitioning Coreset Generation

Final Year Project Report

MEng Computer Science

Submission Date: 22 July 2025

Student ID: 20017758

Supervisor: Dr. Ran Ben Basat

*This report is submitted as part requirement for the MEng Degree in Computer Science at UCL. It is substantially the result of my own work except where explicitly indicated in the text. The report may be freely copied and distributed provided the source is explicitly acknowledged.

Abstract

This project investigates whether grid-based partitioning methods can be used to construct effective coresets for accelerating k-means clustering in large, low-dimensional datasets. While traditional coreset approaches often rely on complex sampling techniques or theoretical sensitivity bounds, this work explores a more intuitive and geometric strategy: partitioning the data space into uniform or adaptive grids to form compact, weighted summaries of the original dataset.

Three coreset methods were implemented and evaluated: a static Grid Mean baseline, an adaptive Recursive Grid using recursive spatial subdivision guided by scoring heuristics, and a Guided Grid that incorporates intermittent feedback from k-means clustering to refine its grid adaptively. Each method was designed to be highly tunable, and a variety of scoring heuristics, grid start resolutions, and guidance parameters were tested to optimize performance.

Extensive experiments were conducted across synthetic and real-world datasets (Blobs, Covertype, and US Census) in both 2D and 3D. Evaluation focused on clustering accuracy (measured as the final k-means cost on the original dataset) and total runtime, including coreset construction. Results show that Recursive Grid consistently outperforms other methods in accuracy, often achieving near-baseline cost with substantial time savings. Grid Mean provided a fast, reliable alternative, while Guided Grid did not perform as well as expected cost-wise and having the highest runtime. Random sampling served as a naive baseline, highlighting the consistent advantages of grid-based strategies.

The analysis affirms that grid-based coresets offer a meaningful cost-runtime trade-off for k-means clustering in low dimensions. Despite implementation limitations in Python, the results demonstrate the potential of these methods and suggest promising directions for further optimization and hybridization.

Contents

1 Introduction **4**

2 Literature Review **6**

2.1 Clustering and the k-means Algorithm 6

2.2 Coresets: Concept and Motivation 6

2.3 Classical Coreset Methods for k-means 7

2.4 Grid-Based and Spatial Partitioning Approaches 7

2.5 Grid-Based Coresets: Connecting Grids with Coreset Ideas 8

2.6 Gaps and Opportunities 9

3 Design and Development **10**

3.1 Motivation and Design Objectives 10

3.2 Initial Design: Static Grid Coreset 10

3.3 Practical Challenges with Target Size 11

3.4 Problems with a Static Coreset: Non-Uniform Densities 11

3.5 Recursive Grid: Local Adaptive Refinement 12

3.6 Guided Grid: Refinement with Clustering Feedback 13

3.7 Scoring Heuristics: Core Principles 13

3.8 Rejected Extension: Batched Splitting 14

3.9 Final Thoughts on Design & Development 15

4 Methodology **16**

4.1 Methodological Scope and Objectives 16

4.2 Implementation Summary 16

4.3 Testing Framework and Experimental Pipeline 21

4.4 Sanity Checks and Verification 22

4.5 Preliminary Parameter Testing 24

4.6 Baseline Methods 26

4.7 Datasets 27

4.8 Experiment Design 28

4.9 Hardware and Environment 29
4.10 Implementation Constraints 29

5 Results and Analysis **31**

5.1 Overview and Evaluation Criteria 31
5.2 Recursive Grid Evaluation 31
5.2.1 Score Function Comparison 31
5.2.2 Start Factor Analysis 34
5.3 Guided Grid Evaluation 35
5.3.1 Score Function Comparison 35
5.3.2 Start Factor Analysis 37
5.4 Full Comparison Across Methods 40
5.4.1 Cost Comparison 40
5.4.2 Time Comparison 42
5.4.3 Summary of Trade-offs 43
5.5 Discussion 44
5.6 Limitations and Future Testing Extensions 45

6 Conclusion **47**

Appendix **50**

A Additional Figures **50**

A.1 Recursive Results Cost Outliers 50
A.2 Recursive Results Time Outliers 50
A.3 Guided Results Time Outliers 51
A.4 Full Results Cost Outliers 52
A.5 Full Results Time Outliers 53

B Project Plan **55**

C Interim Report **59**

1 Introduction

Clustering is a fundamental task in data science and unsupervised machine learning that helps uncover hidden structure in data by grouping similar points together without the need for predefined labels. The ability to cluster data effectively has become increasingly important in the modern world, where organisations and researchers routinely collect large datasets in fields such as marketing, social network analysis, genomics, and image processing. k-means [19] remains one of the most popular clustering methods and is widely used due to its simplicity, ease of implementation, and generally strong empirical performance. By minimising the sum of squared distances between data points and their nearest cluster centroid, k-means offers an intuitive way to summarise complex datasets with a small set of representative centers.

Despite its popularity, k-means comes with a critical practical limitation: its computational cost scales linearly with the number of data points, the number of clusters, the number of dimensions, and the number of iterations needed to converge. For small and medium-sized datasets, modern implementations such as the k-means++ initialisation [3] and efficient libraries like *scikit-learn* [21] make clustering feasible for everyday tasks. However, when working with millions of points, even in relatively low-dimensional contexts such as 2D or 3D geospatial data, environmental models, or dense point clouds, linear scaling quickly becomes a significant bottleneck. Running multiple restarts with different initialisations, which is standard practice to improve clustering results, only adds to this computational burden.

One widely studied approach to address this scalability challenge is the use of coresets [12]. A coreset is a carefully chosen, small, weighted subset of the original dataset that approximates the cost function of the full data within a provable bound. The practical idea is straightforward: if a good coreset can be constructed, then running k-means on this smaller representative subset should yield a solution close to what would be found by running k-means on the full dataset, but at a fraction of the computational cost. Coresets have shown great promise in various settings [22], but many state-of-the-art techniques rely on sophisticated sampling strategies, such as sensitivity-based or importance sampling, which often require multiple passes over the data or complex probability computations. While theoretically sound, these methods can be cumbersome to tune and costly to implement in practice, particularly for straightforward low-dimensional spatial data, where local geometric structure can be exploited more directly.

An alternative perspective is to leverage the inherent spatial structure of low-dimensional data through grid-based or hierarchical partitioning methods. These methods, with a long history in computer graphics, spatial databases, and computational geometry, group nearby points into cells, whether using a static grid or an adaptive tree structure such as a quadtree [23], and summarise each cell using simple statistics like its centroid and point count. The key idea is that points which are close together often contribute similarly to the clustering cost, so representing them collectively can reduce redundant computation while retaining the essential structure of the data. This makes grid-based summarisation particularly appealing for 2D and 3D datasets, where such geometric locality is natural and intuitive. Moreover, adaptive variants of these techniques can allocate more detail where it is most needed, such as near cluster boundaries or in regions with high local variation.

Motivated by these considerations, this project explores the question of whether grid-based partitioning coresets can provide a practical balance between computational speed and clustering accuracy specifically for low-dimensional datasets. It asks: *can grid-based partitioning coresets provide a meaningful trade-off between clustering accuracy and computational efficiency for k-means clustering in low dimensions?* Answering this question requires not only designing suitable methods but also systematically evaluating them to understand their practical trade-offs.

To address this question, three related but distinct grid-based coreset construction methods were developed. The first, called *Grid Mean*, serves as a baseline: it overlays a static, fixed-

resolution grid on the data space, groups all points falling within each cell, and represents each cell by its mean and the number of points it contains. This method is simple and fast but does not adapt to local variation in point density or spread. The second method, called *Recursive Grid*, adds adaptivity by starting with an initial coarse grid and then refining it iteratively. It uses a scoring function based on properties such as the number of points in a cell, the spread or variance of those points, or other local measures to prioritise which cells should be split. A priority queue ensures that the highest-scoring cell is always split next, continuing until a target coreset size is reached. The third method, *Guided Grid*, builds on the Recursive Grid by introducing a feedback loop: after constructing an initial coreset, k-means is run on the coreset to generate approximate cluster centers, which are then used to refine the scoring function. For example, a cell that contains points from multiple clusters or where assignments are ambiguous can be prioritised for further splitting. This feedback process is repeated intermittently to refine the coreset where the representation is weakest.

Together, these three methods form a progression from a simple, static baseline to more sophisticated adaptive techniques that aim to direct computational effort where it is most likely to reduce clustering error, while staying practical for real 2D and 3D data. To test their effectiveness, the methods are evaluated against two standard baselines: the classic k-means algorithm running on the full dataset with k-means++ initialisation, and a simple random sampling coreset that selects a fixed number of points uniformly at random. The main metrics for comparison include the time required to construct each coreset, the time taken to cluster it, and the final k-means *cost* (defined as the sum of squared distances from each point to its nearest computed centroid) as a measure of clustering quality.

The main goal of this project is to design, implement, and compare these grid-based coreset methods to better understand whether they can achieve a meaningful trade-off between speed and accuracy for low-dimensional datasets. By systematically testing the methods on synthetic and real 2D and 3D datasets of varying size, the project aims to identify the situations where grid-based partitioning coresets are most effective, the limitations of the current implementations, and directions for future improvement.

The remainder of this report is structured as follows. The next section is a literature review of the background concepts related to k-means clustering, coreset construction, and grid-based spatial partitioning. This is followed by a description of the design and development process, including the rationale for each method, key practical considerations, as well as rejected extensions. The methodology section then presents the final algorithms, testing framework, preliminary testing and sanity checks, experimental setup, and implementation constraints. This is followed by the results and analysis, which thoroughly compares the parameterized versions of the recursive and guided methods, then compares their most optimal versions with the baselines and discusses their effectiveness, as well as the project's limitations and possible future extensions. The report concludes with a summary of the key findings, limitations, and future extensions.

2 Literature Review

2.1 Clustering and the k-means Algorithm

Clustering is a core tool in unsupervised learning, providing a way to discover hidden structure in unlabeled data by grouping similar points together. Some of its various applications include image processing, customer segmentation, anomaly detection, and bioinformatics. Among the many algorithms developed for this purpose, k-means [19, 18] remains one of the simplest and most widely used methods.

The idea behind k-means is conceptually straightforward: given a set of data points and a desired number of clusters k, the algorithm partitions the data by finding cluster centers that minimise the total sum of squared distances from each point to its nearest center (i.e. minimizing clustering cost). In practice, the algorithm iteratively reassigns each point to its closest center and then recomputes each center as the mean of its assigned points, repeating this process until convergence.

Despite its elegance and ease of implementation, the standard k-means algorithm is not without drawbacks. Its runtime scales linearly with the number of points, clusters, and dimensions, making it increasingly expensive as datasets grow larger and higher in dimensionality. Moreover, the algorithm is sensitive to the initial placement of centers and may converge to poor local minima. The widely used k-means++ seeding method [3] addresses this by providing a probabilistic strategy for spreading out initial centers, leading to better clustering performance on average. Nonetheless, multiple restarts with different seeds are common practice to reduce the risk of poor solutions, which further increases computational cost.

These limitations are particularly acute when data cannot fit into memory, must be processed in near-real time, or comes in continuous streams. In such scenarios, full-data clustering can be infeasible. The challenge then becomes how to approximate the k-means objective efficiently without sacrificing too much accuracy, a question that leads naturally to the concept of coresets.

2.2 Coresets: Concept and Motivation

A coreset is a small, carefully selected subset of a dataset that approximates a given objective function within a provable error bound. When applied to k-means, the idea is simple but powerful: a small, weighted summary that represents the structure of the full data is clustered instead of the entire dataset.

This yields two major benefits. First, they dramatically reduce the size of the problem, which can cut down both runtime and memory requirements. Second, they come with theoretical guarantees: under reasonable conditions, clustering the coreset provides an approximation within $(1 \pm \epsilon)$ of the true optimum, where ϵ can be made arbitrarily small at the cost of a slightly larger coreset [16, 12].

The coreset principle has become an influential idea in scalable machine learning, enabling algorithms that would otherwise be computationally prohibitive. In practice, however, constructing a good coreset is not trivial. The challenge is to summarise the data without spending more time on the summarisation than on simply running k-means directly. This balance between theoretical guarantees and practical efficiency has shaped much of the recent research on coresets.

2.3 Classical Coreset Methods for k-means

The simplest coreset construction for k-means is uniform random sampling. This method draws a small subset of points at random from the full dataset and assigns each point a weight based on its sampling probability. While easy to implement and useful as a baseline, uniform sampling does not account for the fact that not all points contribute equally to the clustering cost. It may underrepresent small or distant clusters and can fail to capture the fine-grained structure in non-uniform or highly variable data.

To improve on this, researchers have developed more informed sampling methods that take each point's importance into account. One influential approach is sensitivity sampling, which measures how much influence a point can have on the clustering cost. Intuitively, points that lie on cluster boundaries or far from other points can have a disproportionate effect on the total cost, so they deserve higher sampling probabilities. Har-Peled and Mazumdar [16] and Langberg and Schulman [12] formalised this idea and showed that it is possible to bound coreset size using sensitivity analysis and the VC-dimension of the data space.

In practice, however, computing exact sensitivities is challenging because it requires knowledge of the optimal clustering solution, the very thing the coreset is meant to approximate. Practical methods therefore rely on approximate heuristics, coarser clusterings, or bicriteria relaxations, where the algorithm uses more than k centers or allows a looser cost approximation to guide the sampling. Kanungo et al. [17] and subsequent studies have shown that such bicriteria relaxations can provide useful practical trade-offs, yielding smaller coresets with acceptable accuracy.

Another significant branch of coreset research addresses streaming data. In many modern applications, datasets arrive continuously and cannot be stored or processed in full. This led to the development of merge-and-reduce frameworks [16], which build coresets incrementally by splitting incoming data into manageable chunks, constructing local coresets for each chunk, and merging them hierarchically.

StreamKM++ [1] is a notable example. It extends the classic k-means++ seeding to a streaming setting by summarising chunks with a grid structure and combining them using merge-and-reduce operations. BICO [7] takes a similar approach by integrating clustering features from the BIRCH algorithm with coreset theory, producing a hierarchical tree of local summaries.

Recent work has sought to unify and simplify these ideas. The Unified Framework [11] and the New Coreset Framework [8] provide modular recipes for building coresets under different constraints, combining sensitivity sampling, merge-and-reduce, or approximate heuristics as needed. Lightweight coresets [4] goes further, relaxing the full theoretical guarantees in favor of faster, simpler constructions that still perform well in practice.

Together, these methods demonstrate that coresets are a robust tool for compressing large datasets for k-means, balancing computational effort and accuracy. However, all sampling-based approaches share a limitation: constructing the coreset itself can be computationally demanding, especially if multiple passes over the data are required to estimate sensitivities or local costs.

2.4 Grid-Based and Spatial Partitioning Approaches

While much of the literature focuses on sampling points based on probability or sensitivity, another practical approach leverages the spatial structure of the data directly. Grid-based and hierarchical partitioning methods take advantage of the fact that points that are close in space are likely to belong to the same cluster and therefore can often be grouped without significant loss of information.

One of the earliest systematic uses of grids for summarising spatial data is the STING framework [25]. STING divides the data space into a hierarchy of rectangular grid cells and stores simple statistics for each cell, such as the number of points, mean, and variance. This hierarchical arrangement enables region-based queries and basic clustering operations to be answered efficiently using cell summaries instead of scanning the entire dataset.

In higher-dimensional settings, CLIQUE [2] extended this idea by partitioning each dimension into intervals, forming a multi-dimensional grid to find dense units. CLIQUE demonstrated that grid-based partitioning can tame the curse of dimensionality to some extent by focusing only on relevant subspaces where clusters exist.

Related ideas underpin classic data structures like quadtrees [23], which recursively subdivide space into quadrants, refining representation only where data density or variation warrants it. Similarly, kd-trees adaptively split space along coordinate axes to group nearby points, making them useful for nearest-neighbour queries and approximate clustering.

Even density-based clustering methods like DBSCAN [10] rely on spatial indexing, using region queries to detect clusters of arbitrary shapes based on local density.

While these techniques were not originally designed as coresets, their underlying principle aligns well with the goal of reducing redundant computations: representing nearby points with simple cell-level summaries, compressing data naturally and intuitively.

2.5 Grid-Based Coresets: Connecting Grids with Coreset Ideas

While traditional spatial partitioning methods like STING or quadtrees were not originally framed as coresets, recent research has shown that combining grid structures with coreset principles is a powerful way to build fast, practical approximations for k-means.

One early example of this hybrid thinking is StreamKM++, which processes streaming data by assigning points to grid cells and representing each cell with a centroid and weight [1]. Similarly, BICO [7] summarises local regions with small, mergeable clustering features, effectively behaving like a dynamic grid coreset.

More modern approaches develop this idea in more flexible ways. The MARCO algorithm [6] uses a recursive grid-like subdivision with feedback from k-means runs, splitting only those blocks that contain mixed cluster assignments. This idea of refining a static grid adaptively based on cost improvement directly motivates similar strategies used in the Boundary Weighted K-means (BWKM) framework.

The Resolution Coreset [13] takes this further by formalising grid coarsening for data that naturally fits a voxel grid. This is especially relevant for image data and 3D point clouds, where the data's inherent spatial structure aligns well with a grid representation.

Recent work by Fiedler et al. is complemented by other modern studies. The Geometric Coreset [14] highlights how simple geometric partitioning, such as overlaying a grid or using space-partitioning trees , can yield small, provably good coresets by carefully selecting representatives. Similarly, Fast k-means using Coresets [15] demonstrates that simple grid-based and tree-based constructions can significantly reduce runtime for approximate clustering without needing complex sensitivity scores.

A practical appeal of these designs is that they often rely on elementary operations, such as counting points in cells, computing local means, and deciding when a cell needs more splits, which are easy to implement and tune. For example, Fast k-means shows that combining grid-like partitioning with well-chosen sampling heuristics produces coresets that are fast to build yet

competitive with more elaborate, purely probabilistic methods.

The core insight is that the geometry of the data can do much of the work. When points cluster spatially, a grid cell's centroid naturally acts as a local representative. Hierarchical structures like quadtrees or kd-trees allow this representation to adapt to non-uniform densities by refining only where detail matters.

2.6 Gaps and Opportunities

Despite these advances, there are still open questions around how best to build grid-based coresets that are truly competitive for large, complex datasets. Fixed-resolution grids risk over-simplifying dense regions and wasting computation in sparse areas. Adaptive approaches like MARCO, BWKM, and modern geometric coreset frameworks help, but they introduce design questions: *How should cells be scored for splitting? How deep should recursive splitting go? When should partial k-means runs be triggered to guide refinement?*

This motivates this project's practical design: starting with a simple static grid, adding recursive refinement with clear scoring heuristics, and testing whether using k-means feedback can guide splitting in a way that balances speed and approximation accuracy for real-world 2D–3D datasets.

In doing so, this project draws inspiration from frameworks like the MARCO algorithm [6], Geometric Coresets [14], and practical quadtree-based ideas like STING [25] that blend local geometry with simple coreset theory. The aim is to clarify when these hybrid, geometry-aware coresets provide real computational benefits over purely random based sampling for practical large-scale clustering.

3 Design and Development

3.1 Motivation and Design Objectives

This project was motivated by a simple but practical goal: make k-means clustering [19] faster for large, low-dimensional datasets, especially in common scenarios like 2D or 3D spatial data. Although k-means is widely used for its simplicity and decent performance, it does not scale well when the dataset is large or when multiple initialisations are needed for good results. Even for straightforward 2D or 3D point data like maps, point clouds, or remote sensing grids, millions of points can make repeated k-means runs a real bottleneck.

Rather than relying on sensitivity sampling or complex multi-pass schemes that add extra computational work or are tricky to tune in practice. This project set out to explore whether a more intuitive coreset generation method could work well for low-dimensional data, using the spatial structure of the data itself as the main tool for building a coreset.

The core design aims were:

- Keep the method easy to understand and explain, with clear geometric intuition.
- Make it adaptive, so more detail is allocated to parts of the data that need it.
- Keep the trade-offs transparent: users should be able to adjust runtime vs. accuracy as needed.
- Test the idea systematically to see when simple grid-based partitioning works well, and when it does not.

3.2 Initial Design: Static Grid Coreset

The first version was built around the simplest possible version of a grid-based coreset: lay down a fixed grid on the data space, group all points in each cell, and replace the cell's points with a single representative and a weight.

The very first attempt, which became known as the Grid Center Coreset, used the geometric center of each grid cell as the representative. The logic was that this would be extremely quick to build: once the grid size was chosen, assigning points to cells and counting them is trivial. The expectation was that this would make both the coreset construction and any later point assignment steps fast.

An idea early on was to also factor in assignment time (time to assign each datapoint their closest center) as a metric, as this would allow to factor in one of the significant advantages of this method over sampling based methods, by taking advantage of the fact that once each Grid Center coreset representative point is clustered, its final cluster label could be propagated back to all the original points in the same cell. In theory, this should have saved time by skipping the final pointwise distance checks for cluster assignments. But practical tests showed this was not as helpful as expected. In Python, doing this label look-up using dictionaries or loops turned out slower than a straightforward NumPy [20] distance calculation for each point. Also, simply assigning all points in a cell to the coreset's cluster center did not always guarantee that each point was truly closest to that center, significantly hindering cost. Therefore the idea of using Assignment time as a metric was scrapped.

A strict upgrade to Grid Center became clear during the initial implementation: instead of using the fixed grid cell center, each cell's points could be averaged to find their true mean and

would an improved representative of the cell. This slight adjustment birthed Grid Center, and came with a significantly improved clustering cost with almost no extra computation.

The static grid approach is simple and quick to build, and it naturally uses the data's local structure. But as the project progressed, it became clear that even this simplicity brings up practical questions about how to pick grid size and whether a single fixed grid could handle uneven or clustered data well.

3.3 Practical Challenges with Target Size

While the Grid Mean method worked well for a quick test, one issue showed up early during experiment planning: different coreset methods naturally produce coresets of different sizes, and comparing them directly would not be fair. Larger coresets often reduce final clustering cost, but they also take longer to build and cluster, so fixing a clear target coreset size was essential for a fair test.

Unlike random sampling, where you can just pick a sample size, the static grid's coreset size is not set directly. It depends on how many cells end up with at least one point. This depends on the grid resolution and the local density of the data, which can vary a lot in real-world spatial data.

To work around this, several ideas were researched and tested:

Histogram Approximation: Inspired by spatial sampling ideas such as those discussed by Brus in *Spatial Sampling with R* [5], this approach aimed to guess how many non-empty cells a given grid size would produce. The theory works fine if the data is evenly spread, but for real clustered data, it often under- or over-shot badly.

Density Heuristics: Another idea was to estimate grid size using rough density indicators like nearest-neighbour distances (following concepts in Dixon's *Nearest Neighbor Methods* [9]) or dispersion indexes. The hope was that these quick checks could adjust grid size to hit the target coreset size more reliably. But like the histogram method, these heuristics worked well for some datasets and failed badly on others.

Binary Search: A more direct engineering fix was to run a coarse grid, count the non-empty cells, adjust the grid resolution up or down, and repeat until the coreset size matched the target. Some simple bounding logic, like using the dataset's bounding box or local variance, helped speed up the search. But each check still needed a full scan of the dataset, which defeated the point of having a cheap-to-build coreset.

All of these tests showed that for the static grid method, picking a precise coreset size was not practical without adding lots of extra work. The solution was to treat the grid size as a user setting, something that would be chosen ahead of time in real-world use. For testing, any grid size tuning needed to hit a target size was done separately and not counted in the measured runtime.

3.4 Problems with a Static Coreset: Non-Uniform Densities

Beyond the target size issue, the static grid method also showed a natural weakness: it does not adapt when data is non-uniform or clusters overlap awkwardly with the grid lines. For evenly spread points, a fixed grid does well. But when clusters are dense and separated by sparse areas, the grid wastes cells on empty regions while providing too little detail where it is actually needed.

To solve this, an early idea was to refine the grid's resolution step by step. This strawman refinement approach started with a coarse grid, built a coreset, ran k-means to see how close it was to the full-data cost, then increased the grid resolution and repeated until the error dropped below a set level.

This seemed reasonable in theory but quickly hit practical problems:

- Running k-means multiple times just to check cost defeated the time-saving benefit of having a coreset.
- Subdividing the whole grid added unnecessary detail in sparse regions.
- For complex cluster shapes, the improvement flattened out quickly, wasting effort.

Although this idea was dropped, it showed that adaptivity was still worth exploring, just not through global refinement. Instead, the solution was to split only where it mattered.

3.5 Recursive Grid: Local Adaptive Refinement

The limits of the static grid made it clear that some level of adaptivity was needed, but without falling back on expensive global refinement. The Recursive Grid was designed to fill this gap by refining the grid only where it makes the biggest difference.

The idea is straightforward: instead of splitting all cells equally, the method focuses effort on parts of the data that are dense, spread out, or likely to contain mixed cluster regions. The first version worked like a classic quadtree in 2D (or octree in 3D). The entire dataset began as one large cell. If that cell held more points than a set threshold, or its spread was too large, it was split into child cells. Each new cell was then tested and split again if needed. This recursive splitting continued until all cells met the stopping condition or the coreset reached its target size.

However, this pure recursive version ran into problems. Starting with a single huge cell meant that the first few splits handled millions of points at once, so the early stages dominated the runtime. It also led to unnecessary splits in parts of the space that contributed little to the clustering cost, simply to satisfy basic thresholds.

To fix this, the design switched to a priority-based approach. Instead of splitting every cell that crosses a threshold, each cell is scored using a heuristic that measures how useful splitting it would be. A *priority queue* keeps track of all candidate cells, always selecting the one with the highest score to split next. When a cell is split, its child cells are scored too and added back to the queue. This cycle repeats until the coreset reaches the desired number of representatives. By focusing only on the cells that matter most, the method naturally concentrates detail in regions where it improves the clustering cost most effectively, while leaving simple areas untouched.

Additionally, to avoid wasting time on the first big split, the method does not start with one giant cell but instead uses a coarse Grid Mean pass to create an initial set of moderate-sized cells. From there, the priority-driven splitting adds detail only where needed. Because the score can be based on different heuristics like population, variance, or inertia, the refinement can adapt to different kinds of data.

By combining the strengths of a static grid with adaptive local refinement, the Recursive Grid shows how a simple priority system can make grid-based coresets practical and flexible for non-uniform, clustered datasets.

3.6 Guided Grid: Refinement with Clustering Feedback

Even with local splitting, purely geometric rules can miss subtle details. Clusters often do not line up neatly with grid edges, so relying only on local density or spread can leave gaps where cell means blur multiple cluster regions.

The next design step was to test whether feeding back extra information from k-means itself could guide refinement. This led to the Guided Grid, which builds on the Recursive Grid but adds a feedback loop.

After an initial coreset is built with recursive splitting, the algorithm runs a partial k-means on the coreset to find a rough estimate of the cluster centers. These provisional centers are then used directly in the scoring of cells: each cell is checked to see how its points relate to the current cluster centers, for example, whether the points mostly belong to just one cluster or are split across multiple clusters. A cell where all points fall clearly under one cluster is likely fine as it is, but if a cell's points are mixed, or cluster assignments are uncertain, the scoring function gives it a higher priority for splitting.

This cluster-informed scoring means the method no longer relies purely on local spread or point counts but also on how well each cell matches the current view of the cluster layout. The result is that extra splits happen where they can reduce boundary errors or capture tricky shapes that simple local scores alone might overlook.

The feedback is run intermittently during refinement, so the coreset can adapt as it grows. The balance between how often k-means runs and how strongly this feedback affects scoring can be adjusted, more frequent updates can help accuracy but add to total runtime.

This idea was inspired by methods like BWKM [6], which use provisional cluster boundaries to decide where to refine detail. The Guided Grid keeps this concept simple: it blends cluster-center feedback with local scores to target splits where they matter most, giving the coreset an extra edge for real-world data with complex cluster overlaps.

3.7 Scoring Heuristics: Core Principles

The scoring heuristics are the core tool that makes the Recursive Grid and Guided Grid work. They decide where the coreset adds more resolution, and where it doesn't need to.

Instead of splitting cells blindly, each cell is given a score that shows how much splitting it could help lower the final clustering cost. Some scores only look at local geometry, like how many points are inside a cell, or how spread out they are. Others, used in the Guided Grid, mix in cluster feedback from partial k-means runs to flag cells where local structure alone might miss hidden cluster edges.

Choosing which score to use, and how strongly to weight cluster feedback if used, directly shapes how accurate and efficient the coreset will be. Together, these heuristics give the methods a way to adapt automatically to different datasets and trade off between speed and approximation quality.

- **Population:** Scores a cell by simply counting how many points it contains. Heavily populated cells have the biggest impact on clustering cost because they stand in for more of the dataset. Splitting large cells first ensures the coreset better represents dense clusters instead of wasting resolution on sparse areas.
- **Variance:** Measures how spread out the points in the cell are by calculating coordinate-wise variance. High variance suggests the cell covers multiple local features or crosses a

cluster edge. Prioritising high-variance cells means the coreset adds detail where a single mean would hide real differences.

- **Inertia:** Calculates the sum of squared distances between each point in the cell and the cell's mean, directly matching the k-means cost. If inertia is high, the cell's representative may poorly reflect the local cost structure, so splitting helps reduce the final k-means error.
- **Mean Absolute Deviation (MAD):** Uses the average absolute distance of points from the cell's mean instead of squared distances. This is similar to variance but is less sensitive to outliers, making it useful for catching spread in skewed or irregular cell shapes without overreacting to single distant points.
- **Diameter:** Estimates the maximum distance between any two points inside the cell. A large diameter can flag cells that cover long, thin or oddly shaped regions, a sign that more splits could stop the cell's mean from crossing over multiple clusters.
- **Impurity (Guided):** Checks how mixed a cell's points are in terms of provisional cluster labels from intermediate k-means runs. A cell where many points belong to different clusters suggests a boundary region. Splitting high-impurity cells focuses detail where clusters overlap.
- **Local Error (Guided):** Estimates the local k-means cost for points in the cell, using current provisional centers. If local error is high, the cell likely hides structure that a single mean can't capture well, so splitting should help lower total cost.
- **Margin (Guided):** Looks at how strongly each point "prefers" its assigned cluster compared to its next-best option. A small margin means points in the cell are uncertain about where they belong, splitting such cells can clarify these fuzzy boundaries.
- **Label Entropy (Guided):** Uses Shannon entropy to measure how evenly the cell's points are distributed across clusters. High entropy means more confusion about cluster assignment, which is a good signal that extra detail will help.

Together, these heuristics make it possible to tune the refinement process, from simple rules that need only local stats to more advanced checks that blend in clustering feedback. Choosing the right score could shift the balance between speed and accuracy to match different datasets.

3.8 Rejected Extension: Batched Splitting

During development, one natural question was whether the splitting loop could be made faster by handling multiple splits at once. The idea was *batched splitting*: instead of popping just the top cell from the priority queue each time, the algorithm would pop a batch of top-scoring cells and split them all in parallel.

In theory, this should reduce the number of queue operations, which can add up when there are thousands of cells to manage. It could also pair well with the Guided Grid, since the batch size could be aligned with the cluster feedback loop, running k-means after each batch instead of after every single split.

But in practice, this idea did not deliver useful improvements. Python's lack of true parallelisation for this type of data processing means that splitting multiple cells in "parallel" still happens step by step under the hood. Combined with the general overhead of Python's dynamic data structures, this cancels out much of the expected speed gain.

More importantly, batching introduces an inherent trade-off in accuracy: when multiple cells are split at once, the algorithm spreads refinement effort across several moderately important

areas instead of always picking the single most impactful cell first. This weakens the priority queue's strength, focusing detail exactly where it reduces cost the most. In tests, this small loss of precision slightly degraded coreset quality, while the runtime savings were minimal.

Given these practical drawbacks, the batched variant was left out of the final methods. While a low-level compiled version using true parallelisation, for example, with C++ threads, might make batched splitting worthwhile, in a Python prototype, the balance of complexity, small time gains, and accuracy loss did not justify it.

3.9 Final Thoughts on Design & Development

This design process shows how the project moved step by step from a simple static grid coreset to more adaptive and guided approaches that respond better to real cluster structures. Each method builds on the last, addressing limits like non-uniform densities and hidden cluster boundaries while staying simple enough for practical use on low-dimensional data. The scoring heuristics and feedback loop give clear ways to balance *runtime and accuracy*, letting users trade extra coreset construction time for tighter clustering results when it matters. Together, these choices form a practical toolkit for testing whether grid-based partitioning coresets can really help speed up k-means without losing too much quality, a question that the next sections set out to answer in practice.

4 Methodology

4.1 Methodological Scope and Objectives

To evaluate the effectiveness of the grid-based coreset methods, a structured and reproducible testing framework was developed. The experimental setup is designed to compare three custom-designed coreset algorithms: *Grid Mean*, *Recursive Grid*, and *Guided Grid*; against two widely accepted baselines: standard full-data k-means and random sampling.

Each method is tested under controlled conditions using a consistent evaluation pipeline. The focus is on measuring three primary outcomes:

- The time required to construct the coreset,
- The runtime of k-means clustering applied to the coreset,
- The clustering cost (inertia i.e. the sum of squared distances between data points and their nearest cluster centroid) evaluated on the full dataset.

Tests are conducted across multiple datasets (both synthetic and real-world), varying numbers of clusters k, and a range of coreset sizes. The algorithms are assessed in both two and three dimensions, reflecting the project's goal of improving efficiency for spatial or geometric clustering problems in low-dimensional settings.

In preparation for final testing, multiple phases of development, verification, and parameter filtering were carried out:

- The implemented methods were first validated through sanity checks and controlled test cases.
- A round of preliminary testing was used to identify ineffective or redundant parameter settings, narrowing the scope of the final experiments.
- A fixed set of parameters and scoring heuristics was then used for large-scale evaluation across all datasets and configurations.

The methodology includes a detailed description of the algorithm implementations, scoring strategies, evaluation metrics, testing framework, datasets used, and hardware setup. Limitations related to computational resources and implementation constraints are also acknowledged.

This structured experimental design ensures that performance comparisons between methods are fair, replicable, and informative with respect to the central research question: *can grid-based partitioning coresets provide a meaningful trade-off between clustering accuracy and computational efficiency for k-means clustering in low dimensions?*

4.2 Implementation Summary

All three coreset construction methods: *Grid Mean*, *Recursive Grid*, and *Guided Grid*, were implemented in Python using an object-oriented architecture to enable modularity, reusability, and flexible experimentation. Each method inherits from a common *AbstractCoreset* class, which defines a standard interface for coreset generation and ensures consistency across testing and implementation.

The `AbstractCoreset` base class defines:

- A *constructor*, which is overridden in each subclass to accept the relevant tunable parameters for that method (e.g. `grid size, scoring function, or feedback interval`). This allows coreset objects to be configured in advance and reused across multiple test scenarios.
- A method called `get_coreset(data, k_clusters, seed)`, which takes three inputs:
  - the dataset (as a NumPy array),
  - the number of target clusters k,
  - and a random seed for reproducibility.

These three inputs are passed in automatically by the testing framework during execution, and not handled manually by the user. The only parameters explicitly specified by the user are those passed into the constructor when the coreset object is created. This distinction cleanly separates the test-time inputs (e.g. dataset, k, seed) from the algorithm configuration, and greatly simplifies the process of comparing methods under controlled conditions.

This structure enables highly flexible experimentation. For example, coresets can be instantiated with different scoring strategies or grid resolutions, while the test framework itself remains unchanged. Once configured, these objects are passed into the evaluation pipeline, which calls `get_coreset()` on them during each test run to generate the actual coreset used for clustering.

Grid Mean

Grid Mean implements a simple, fixed-resolution spatial summarisation technique. The data is first translated so that the minimum coordinate in each dimension is zero or positive. Then, each point is assigned to a grid cell using integer division by a global `grid size` parameter, effectively mapping it into a discrete cell-based space.

A dictionary is used to group points by their grid cell coordinates. For each non-empty cell, a single representative point is calculated as the arithmetic mean of the points it contains. This point becomes a coreset point, and its associated weight is equal to the number of original points it summarises. The pseudo-code for the Grid Mean algorithm is detailed in Algorithm 1

- Parameter:
  - `grid_size`: the side length of each grid cell.

The `grid_size` can be manually chosen or determined algorithmically depending on the application. In this project, it is typically computed using a binary search routine that finds a grid size expected to produce a coreset of roughly the desired size. This process is external to the algorithm itself and is not included in runtime measurements, since it represents an offline pre-processing step to enable fair comparative testing.

Grid Mean serves as a baseline that offers speed and simplicity, but does not adapt to variations in data density or distribution.

Algorithm 1 Grid Mean Coreset

Input: Dataset $\mathcal{D} \in \mathbb{R}^{n \times d}$, grid size $g \in \mathbb{R}$
Output: Coreset points C, weights w
$min \leftarrow \min(\mathcal{D})$
$\mathcal{D}_{\text{shift}} \leftarrow \mathcal{D} - min$
$indices \leftarrow \lfloor \mathcal{D}_{\text{shift}}/g \rfloor$
Initialize empty dictionary *cells*
for all $(i, coord) \in indices$ **do**
    Append i to $cells[coord]$
end for
Initialize $C \leftarrow []$, $w \leftarrow []$
for all $cell_indices \in cells$ **do**
    $P \leftarrow \mathcal{D}[cell_indices]$
    C.append(mean(P))
    w.append($|P|$)
end for
return (C, w)

Recursive Grid

Recursive Grid extends the fixed-grid approach by introducing an adaptive refinement process based on local characteristics of the data. Like Grid Mean, it begins by partitioning the space into a regular grid with an initial *starting grid size*, which is typically estimated using the same binary search strategy. However, rather than summarising each cell immediately, Recursive Grid recursively splits selected cells based on a priority scoring function.

Each non-empty grid cell is represented as a `Cell` object, storing:

- the cell's spatial origin and size,
- the list of points it contains,
- a `score`, computed using a user-defined

Cells containing multiple points are inserted into a priority queue (implemented as a max-heap), which selects the highest-scoring cell to split at each iteration. Splitting is performed by subdividing the cell evenly along all dimensions into 2^d child cells. The refinement process continues until the number of coreset points reaches a fixed *target coreset size*. The pseudo-code for the Recursive Grid algorithm is detailed in Algorithm 2

- Parameters:
  - `starting_grid_size`: controls the resolution of the initial coarse grid.
  - `target_coreset_size`: total number of output coreset points.
  - `score_fn`: a callable function that computes a score based on a cell's internal points. Several scoring heuristics are supported, including:
    - number of points (population),
    - coordinate-wise variance,
    - total inertia,
    - mean absolute deviation (MAD),

* or estimated diameter.

The modular structure of the `score_fn` interface makes it easy to compare different heuristics. Recursive Grid aims to place more resolution where it matters most, such as in dense regions or near cluster boundaries, by prioritising the cells that contribute most to clustering cost if poorly summarised.

Algorithm 2 Recursive Grid Coreset

Input: Dataset $\mathcal{D}$, grid size g_0, target coreset size m, score function s
Output: Coreset points C, weights w

$min \leftarrow \min(\mathcal{D})$
$indices \leftarrow \lfloor (\mathcal{D} - min)/g_0 \rfloor$
Group points into *cell_buckets* by *indices*
Initialize empty *heap* and *single*
for all cell $B \in cell_buckets$ **do**
    Create Cell with score $s(B)$
    if $|B| = 1$ **then**
        Add to *single*
    else
        Add to *heap*
    end if
end for
while $|heap| + |single| < m$ **do**
    Pop highest-score cell C from *heap*
    $children \leftarrow \text{Split}(C)$
    for all $child \in children$ **do**
        $child.score \leftarrow s(child.points)$
        if $|child.points| = 1$ **then**
            Add to *single*
        else
            Push into *heap*
        end if
    end for
end while
Extract centroids and weights from $heap \cup single$
return (C, w)

Guided Grid

Guided Grid builds on Recursive Grid by incorporating feedback from k-means clustering during the coreset construction process. It uses the same grid initialisation and recursive refinement framework, but periodically updates the cell scores based on cluster assignment statistics.

After a user-defined number of splits (based on `km_interval`), the method constructs a temporary coreset from the current set of cells and runs a weighted k-means clustering step. The resulting cluster centers are used to re-score each cell in the priority queue. The pseudo-code for the Guided Grid algorithm is detailed in Algorithm 3

- Parameters:
  - `starting_grid_size`
  - `target_coreset_size`

- `score_fn` (now takes both cell points and current cluster centers), this includes:
  - *Impurity*: the fraction of points in a cell that do not belong to the majority cluster,
  - *Error*: the total squared distance to the nearest cluster center,
  - *Margin*: the average gap between the closest and second-closest centers,
  - *Label entropy*: the entropy of cluster assignments within the cell, normalised to [0, 1].
- `km_interval`: number of splits between successive k-means feedback updates (expressed as a fraction of target size),
- `km_kwargs`: optional keyword arguments for configuring the internal k-means runs,
- `km_reuse_centers`: whether to re-use previous k-means centers for warm-starting.

This method refines coreset construction not only based on geometry, but also on ambiguity or conflict in the evolving cluster structure. Although it introduces additional computational overhead, it improves adaptivity in cases where cluster boundaries are complex or overlapping.

Algorithm 3 Guided Grid Coreset with Cluster Feedback

Input: Dataset $\mathcal{D}$, grid size g_0, coreset size m, clusters k
    Score function $s(\cdot, \cdot)$, km interval I, kwargs κ, reuse_centers (boolean)
Output: Coreset points C, weights w
Initialize *heap* and *single* from initial cell partition using g_0
$next_kmeans \leftarrow 0$, $centers \leftarrow$ `None`
while $|heap| + |single| < m$ **do**
    $next_kmeans \leftarrow next_kmeans - 1$
    if $next_kmeans \leq 0$ **then**
        Build temporary coreset (P, w) from $heap \cup single$
        if reuse_centers **and** $centers \neq$ `None` **then**
            $init \leftarrow centers$
        else
            $init \leftarrow$ "k-means++"
        end if
        Run weighted k-means on P with k clusters using $init$, weights w, and kwargs κ
        $centers \leftarrow$ resulting cluster centers
        $next_kmeans \leftarrow I \cdot m$
        for all $cell \in heap$ **do**
            $cell.score \leftarrow s(cell.points, centers)$
        end for
        Heapify *heap*
    end if
    Pop highest-score cell C from *heap*
    $children \leftarrow \text{Split}(C)$
    for all $child \in children$ **do**
        $child.score \leftarrow s(child.points, centers)$
        if $|child.points| = 1$ **then**
            Add to *single*
        else
            Push into *heap*
        end if
    end for
end while
Extract (C, w) from $heap \cup single$
return (C, w)

4.3 Testing Framework and Experimental Pipeline

To ensure fair and consistent evaluation across all coreset methods, a unified testing framework was developed. This framework automates the execution of tests, enforces reproducibility through fixed random seeds, and collects key performance metrics for analysis. Each test run evaluates the end-to-end effectiveness of a coreset method, from construction to final clustering.

Test Execution Flow

Each test is defined by a specific configuration, including:

- A *dataset*,
- A *target number of clusters* k,
- A *target coreset size* T,
- A specific *coreset method* (pre-initialized with parameters), and
- A fixed *random seed*.

For each test:

1. The coreset object is instantiated with all required parameters (e.g. grid size, scoring function, feedback settings).
2. The framework calls the method's `get_coreset(data, k_clusters, seed)` function to generate a coreset. The result is a set of coreset points and their associated weights.
3. K-means clustering is applied to the coreset using scikit-learn's `KMeans` implementation, with `n_init=10` initialisations and *k-means++* seeding. The random seed used for clustering matches the one used for coreset generation.
4. The final clustering *cost* (inertia) is calculated on the full dataset by assigning each original point to its closest center obtained from the coreset.

This workflow allows each method to be evaluated in terms of both its construction efficiency and the accuracy of the resulting cluster solution.

Metrics Recorded

Each test trial records the following key metrics:

- **Coreset construction time**: how long the method takes to generate the coreset,
- **K-means clustering time**: time spent clustering the coreset,
- **Final clustering cost**: total inertia when the learned centers are evaluated on the full dataset.

These metrics provide a balanced view of performance, reflecting both computational efficiency and clustering quality.

Precomputation Step: Grid Size Estimation

For all grid-based methods, the grid or cell resolution significantly affects the resulting coreset size. To facilitate consistent comparisons across different methods and settings, grid sizes were estimated using a binary search routine designed to approximate a target number of non-empty cells.

This estimation procedure is carried out before the test run begins:

- It is computed once per test configuration,
- It is not included in the recorded construction time,
- It simulates a realistic setup step a user might perform prior to execution.

By treating this as an auxiliary step, the measured runtimes reflect the actual execution of the coreset algorithms themselves, independent of hyperparameter tuning.

Seed Management and Reproducibility

All experiments were repeated across **10 distinct seeds**, numbered 0 through 9. These seeds are used consistently across all methods, ensuring that random sampling, k-means initialisation, and any internal randomisation behave identically across configurations.

Using shared seeds across methods enables reliable performance comparisons and reduces variability caused by random factors.

Result Aggregation and Analysis

For each experiment configuration (dataset, method, k, and target coreset size), results are aggregated across the 10 seeds. The following statistics are computed:

- **Mean**: to evaluate average performance,
- **Median**: to reduce sensitivity to outliers,
- **Standard deviation**: to assess consistency across trials.

These aggregate results form the basis for all comparisons and analyses presented later in the report. They allow both individual and cross-method evaluations, enabling a robust understanding of how the proposed coreset techniques perform under varied conditions.

4.4 Sanity Checks and Verification

Before beginning formal testing, a suite of sanity checks were conducted to verify the correctness of the coreset methods and confirm that their internal components behaved as expected. These tests focused on validating scoring logic, parameter effects, data aggregation, and the behaviour of feedback mechanisms during coreset construction.

Sanity checks were performed on the synthetic datasets only, using a fixed cluster count of $k = 20$. For Recursive and Guided Grid, the target coreset size was set to $T = 10{,}000$, while

Grid Mean and grid size estimation were tested with varying target size parameters instead of a fixed size. The goal was not to assess performance, but to confirm logical correctness, control flow, and consistent algorithmic behaviour before proceeding to large-scale evaluation.

To support this process, *logging* was used throughout development to monitor intermediate values during coreset construction. While logging was not part of the timed testing framework, it played a central role in observing how scores were computed, cells were split, and feedback was applied.

Grid Size Estimation Validation

Grid-based methods rely on a binary search routine to approximate a grid resolution that produces a desired number of non-empty cells. This component was tested independently by applying it to several target sizes and verifying the resulting coreset sizes using the Grid Mean method.

Across all runs, the number of non-empty cells fell within the expected bounds, validating the default **1% tolerance** setting. Logging supported this process by reporting the progression of the binary search and highlighting deviations from target cell counts when they occurred, confirming that the bracketing and convergence logic worked as intended.

Grid Mean Behaviour

Grid Mean was tested under varying grid sizes to ensure that its output coreset size and clustering cost changed in predictable ways. As expected, smaller grid sizes yielded more detailed coresets with lower cost, while coarser grids compressed the data more aggressively at the expense of accuracy.

Logging during these tests confirmed that points were correctly assigned to cells, that the number of populated cells matched expectations, and that the resulting centroids were calculated from the intended point groupings. These checks validated both the binning procedure and the mean/weight aggregation logic.

Recursive Grid Checks

Recursive Grid was verified by testing different start factors and scoring functions to ensure they influenced the refinement process as designed. Lower start factors produced more initial cells, enabling faster early iterations, while higher start factors led to longer processing of large initial cells.

Logging was used extensively to track which cells were split and in what order, confirming that higher-scoring cells were consistently prioritised and that score values were computed consistently during splitting. These observations helped validate the core recursive refinement logic.

Guided Grid Feedback Loop

Guided Grid introduces additional complexity through its periodic use of k-means feedback to improve scoring. Sanity checks focused on confirming that feedback was applied at the correct intervals and that it meaningfully influenced which cells were refined.

Tests showed that the interval parameter correctly controlled how often k-means was triggered and that updated cluster centers altered the distribution of cell scores in sensible ways. Logging made it possible to observe the feedback loop in action, from the update of cluster centers to the recomputation of scores and the resulting changes in cell selection. This confirmed that cluster structure was being incorporated into refinement decisions as intended.

4.5 Preliminary Parameter Testing

To keep the final experiment manageable without sacrificing insight, a round of preliminary parameter testing was conducted. These tests were used to evaluate the impact of configurable parameters in the grid-based coreset methods, identify poor-performing configurations, and eliminate combinations unlikely to provide meaningful benefit.

The purpose was not to benchmark methods, but to understand the runtime vs. accuracy trade-offs of different scoring functions and algorithm parameters, and avoid redundant or costly configurations in the full test suite. This step also helped validate that the interaction between parameters did not hide edge cases that could affect performance interpretation.

Scope and Setup

The methods tested were *Recursive Grid* and *Guided Grid*, as they support multiple tunable components. Grid Mean and Random Sampling have fixed behaviour and were excluded from this phase.

Preliminary tests were conducted using:

- All final *datasets*,
- Fixed cluster count: $k = 20$,
- Fixed coreset size: $T = 10{,}000$,
- One parameter varied per test, others fixed to reasonable defaults.

Default values used when not under test:

- *Recursive Grid*: `start factor = 0.3`, `score = population`
- *Guided Grid*: `start factor = 0.3`, `score = error`, `km_interval = 0.1`, `reuse_centers = False`, `max_iter = uncapped`

This setup ensured clarity in parameter impact without introducing unnecessary variability.

Parameters Explored

Recursive Grid:

- **Scoring Functions**: Population, Variance, Inertia, MAD, Diameter
- **Start Factors**: {0.1, 0.3, 0.5, 0.7, 0.9}

Guided Grid:

- **Scoring Functions**: Impurity, Error, Margin, Label Entropy
- **Start Factors**: Same as Recursive
- **Feedback Parameters**:
  - `km_interval`: {0.01, 0.05, 0.1}
  - `reuse_centers`: {True, False}
  - `max_iter`: {2, 5, 20, 100}

These tests were conducted initially with independent parameter sweeps. However, after initial trends became clear, additional cross-combinations were tested to rule out unexpected parameter interactions. For example:

- All four scoring functions were tested with `reuse_centers = True` to confirm generality of earlier findings.
- Similarly, all scoring functions were retested with `km_interval = 0.01` to ensure the observed trends weren't dependent on a specific feedback frequency.

Findings and Parameter Selection

Recursive Grid:

- **Scoring Functions**: Population and variance heuristics showed *slightly better* time–cost trade-offs overall. However, the differences between all five heuristics were relatively small. No single scoring function dominated across all datasets. As a result, all five were retained for full testing.
- **Start Factor**: Higher values (e.g. 0.9) consistently reduced construction time, while lower values made the process slower due to coarse initial cells. The impact on final clustering cost was not reliably correlated with start factor. To balance runtime diversity, the two extremes, **0.1 and 0.9**, were selected for the final experiments.

Guided Grid:

- **Scoring Functions**: Impurity and error tended to yield slightly lower runtimes, but none of the four cluster-aware scores was clearly superior in cost across all datasets. All four were therefore retained for deeper analysis in the final benchmark.
- `km_interval`: This parameter had a strong effect on construction time. Smaller intervals (e.g. 0.01) substantially increased runtime due to frequent k-means updates, yet produced minimal gains in clustering cost. All tested values confirmed this trend, and `km_interval = 0.1` was chosen as the default for the main evaluation.
- `reuse_centers`: While reusing centers slightly improved runtime, it significantly degraded clustering cost across all tested scoring functions. This confirmed earlier results and justified removing this option from further testing.
- `max_iter`: No consistent trend in performance was observed across the tested values {2, 5, 20, 100}. For simplicity, the parameter was left uncapped in the final benchmark.

These tests confirmed that parameter interactions did not hide unexpected effects, and that simplifying the configuration space would not sacrifice generality in conclusions.

Final Configuration Summary

Method	Tunable Parameters	Final Values Tested
Grid Mean	None	Grid size estimated from target size
Recursive Grid	Score function, Start factor	All 5 scores, Start factor $\in \{0.1, 0.9\}$
Guided Grid	Score function, Start factor, `km_interval`	All 4 scores, Start factor $\in \{0.1, 0.9\}$, `km_interval` =

All other options, including reuse of centers and capped iteration counts, were fixed based on preliminary findings and omitted from further tuning. This reduced the number of test cases while preserving meaningful comparisons across key axes of algorithm behaviour.

4.6 Baseline Methods

To contextualise the performance of the proposed coreset methods, two standard baselines were included in all experiments: a full-data run of k-means (*Identity*) and a simple uniform Random Sampling coreset. These baselines serve as reference anchors for evaluating the clustering cost and runtime trade-offs of grid-based approaches.

Both baselines were implemented within the same testing framework as the main methods and follow the exact same procedure for clustering and evaluation. This ensures fair and consistent comparisons across all metrics.

Identity Baseline (Full Data)

The *Identity* baseline applies the standard k-means algorithm directly to the full dataset, with no data reduction or summarisation. This method represents the best-case scenario for clustering accuracy, as it operates on complete information. However, it also incurs the maximum runtime, making it a useful upper bound for computational cost.

In the context of this project, the Identity baseline provides a clear reference point:

- It serves as a cost lower bound, representing the optimal clustering outcome that coresets aim to approximate.
- It also acts as a runtime ceiling, the benchmark that coreset methods strive to beat in terms of speed, ideally while keeping cost close to that of the full-data solution.

Random Sampling Baseline

The *Random Sampling* baseline selects a uniform random subset of the dataset to form a coreset of the same target size $T = k \times m$. No weights are applied, and all selected points are treated equally during clustering.

This method is extremely fast to compute and acts as a simple, naive baseline that does not exploit any spatial or structural properties of the data. While its runtime is effectively negligible, it provides a meaningful cost upper bound, a kind of "placebo" coreset. A well-designed coreset construction method should, at minimum, outperform random sampling in terms of clustering quality.

Its role in evaluation is not to challenge the proposed methods on speed, but to show that they offer a qualitative improvement over trivial subsampling.

4.7 Datasets

To evaluate the grid-based coreset methods under a variety of conditions, experiments were conducted on three datasets, one synthetic and two real-world, each considered in both 2D and 3D versions. The datasets were selected to cover a mix of controlled cluster structure, natural variation, and non-uniform densities, while remaining within the low-dimensional regime targeted by this project.

For the real-world datasets, dimensionality reduction was applied to match the assumptions of the proposed methods and ensure fair comparisons.

Blobs

The *Blobs* dataset is a synthetic benchmark generated using `sklearn.datasets.make_blobs`[1]. It contains **1 million points**, sampled from Gaussian clusters with moderate spread and some overlap, providing a clean and controllable structure for evaluating clustering performance.

To match the values of $k \in \{10, 20, 30, 40, 50\}$ used in testing, a separate version of the dataset was generated for each k, with exactly k cluster centers. Both **2D and 3D versions** were produced, using the same procedure and a fixed random seed (**42**) to ensure consistency and reproducibility across all runs.

Blobs serves as a useful reference for verifying whether coresets preserve well-separated clusters in idealised, low-dimensional scenarios.

Covertype (PCA)

The *Covertype* dataset is a real-world dataset from the **UCI Machine Learning Repository**[2] originally used for forest cover type prediction. It contains **581,012 samples** with **54 attributes**, including cartographic features and ecological measurements.

Because the original dataset is high-dimensional, *Principal Component Analysis (PCA)* was applied to project it into 2D and 3D for evaluation. This preserves the dominant variance components while aligning the data with the dimensionality assumptions of the coreset methods.

Covertype presents a moderately complex clustering landscape, with spatial features that do not always conform to Gaussian assumptions. It allows the grid-based methods to be tested in a more realistic, naturally occurring data context.

US Census (PCA)

The *US Census* dataset consists of detailed demographic and socio-economic records, originally featuring **68 attributes** per record. The version used here contains **2,458,285 samples**, offering a large-scale and high-dimensional real-world dataset.

As with Covertype, *PCA* was used to reduce the dataset to 2D and 3D forms. The projected versions retain complex and non-uniform spatial structure, making them particularly well-suited for testing how well coreset methods handle irregular distributions, imbalanced densities, and nuanced cluster boundaries.

[1] Part of the scikit-learn library for synthetic data generation; see: F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", JMLR, 2011.

[2] UCI Machine Learning Repository: `https://archive.ics.uci.edu/ml/datasets/covertype`

This dataset provides a challenging real-world test case for coreset construction and clustering approximation in large, noisy settings.

4.8 Experiment Design

The final evaluation was designed to benchmark each coreset method, including the proposed grid-based techniques and standard baselines, across a diverse range of clustering scenarios. Each method was tested across multiple datasets, numbers of clusters, and coreset sizes, using a consistent and controlled setup to ensure comparability and fairness.

The values of k (the number of clusters) and m (a multiplier determining coreset size) were chosen to follow the experimental setup used in the 2022 coreset survey by Schwiegelshohn et al. [24], which proposes a standardised structure for evaluating coreset effectiveness. Specifically:

$$k \in \{10, 20, 30, 40, 50\} \quad \text{and} \quad T = k \times m, \quad \text{with} \quad m \in \{50, 100, 200, 500\}$$

This formulation keeps the coreset size proportional to the complexity of the clustering task and enables systematic testing across a wide range of configurations.

For each combination of method, dataset (six in total, spanning 2D and 3D versions of *Blobs*, *Covertype*, and *US Census*), cluster count k, and coreset size T, the experiment was repeated over **10 trials** using fixed random seeds from 0 to 9. These seeds were used consistently across methods to control random sampling, k-means initialisation, and any internal randomness during coreset construction.

During each trial, the following steps were performed:

1. The coreset was constructed using the method under test.
2. K-means clustering was applied to the coreset using scikit-learn's `KMeans`, configured with 10 initialisations.
3. The resulting centers were evaluated on the full dataset by computing the *clustering cost*: the sum of squared distances between each data point and its nearest cluster center.

The following metrics were recorded for each trial:

- **Coreset Construction Time**
- **k-Means Clustering Time** (on the coreset)
- **Clustering Cost** (evaluated on the full dataset)

Only the core algorithmic components, coreset generation and k-means clustering, were included in timing. Auxiliary steps such as grid size estimation (via binary search), dataset loading, PCA projection, coreset object initialisation, and logging were excluded (logging was removed from the coreset code before running the experiments). This isolates the computational effort required by the core methods themselves.

Each result was aggregated across seeds to report the **mean**, **median**, and **standard deviation** for each metric.

The table below summarises which parameters were tested for each method. These configurations reflect the results of the preliminary testing phase, where parameters with minimal or detrimental impact were pruned to reduce the test space while preserving meaningful diversity.

Method	Tuned Parameters	Tested Values
Grid Mean	None (grid size set externally from T)	None
Recursive Grid	Score function, Start factor	5 score functions: population, variance, inert MAD, diameter; Start $\in \{0.1, 0.9\}$
Guided Grid	Score function, Start factor	4 score functions: impurity, error, margin, label tropy; Start $\in \{0.1, 0.9\}$
Random Sampling	None (sampling set externally from T)	None
Identity (Full)	None (uses full dataset)	None

Table 1: Summary of parameter configurations tested for each method.

4.9 Hardware and Environment

All final experiments were conducted on a *MacBook Pro (M3, 18 GB RAM)* running *Python 3.12.9*, using standard scientific computing libraries including *NumPy* and *scikit-learn*. These libraries were used for all array manipulation, clustering, and large scale numerical operations across the entire testing framework.

Due to Python's *Global Interpreter Lock (GIL)*, the language executes most bytecode in a single thread, limiting the ability to parallelise CPU-bound tasks internally. As a result, all coreset methods and clustering operations were run in serial, one process per test execution, without any form of multi-threaded acceleration or parallel processing within a single script.

To improve throughput during the final benchmarking phase, the tests were split into three independent scripts:

- One script handled the *baselines and Grid Mean* tests,
- The second handled *Recursive Grid* evaluations,
- And the third handled *Guided Grid* evaluations.

These scripts were then run in parallel via separate terminal windows, allowing the system to better utilise its multi-core architecture without modifying the single-threaded nature of each algorithm's implementation. This approach ensured reproducible and fair results for each method while improving the overall runtime of the experimental suite.

4.10 Implementation Constraints

While care was taken to ensure consistency and fairness across experiments, several practical constraints shaped the implementation and evaluation of this project.

First, all coreset methods were implemented in *Python*, which introduces inherent performance limitations. Although many performance-critical operations, such as array manipulation and distance computation, were handled using *NumPy* and the Guided Grid method internally reuses *scikit-learn's* optimised k-means, the overall coreset construction logic remains written in pure Python. This results in a mixed implementation, where some components benefit from compiled performance while others remain bottlenecked by Python's interpretive overhead. In

contrast, *scikit-learn's KMeans* is written in *Cython and C++*, and benefits from substantial optimisation. As a result, runtime comparisons between coreset construction and k-means should be interpreted cautiously, since they reflect different underlying levels of software optimisation.

Second, the experiments were executed on a consumer-grade machine (MacBook Pro with M3 chip and 18 GB RAM), which inherently limited computational throughput. The lack of high-performance compute resources constrained the total number of configurations and parameter combinations that could be feasibly explored. In addition, background programs such as a code editor and web browser remained open during testing, and their varying resource usage over time may have introduced further noise into the timing results. While care was taken to minimise interference, such fluctuations are an unavoidable source of variance in desktop-based experiments.

Finally, the computational cost of full testing placed limits on statistical and parameter coverage. Each configuration was repeated using **10 fixed random seeds**, providing a baseline level of repeatability, but additional seeds would further stabilise average measurements. Similarly, the number of parameter combinations tested was reduced to prevent exponential growth in the total runtime. For instance, even with pruning, running the full set of Guided Grid experiments alone required nearly **24 hours**. These constraints made a complete grid search infeasible and narrowed the scope of possible comparisons.

Taken together, these limitations reflect realistic trade-offs in experimental design within a constrained academic setting. While they do not undermine the validity of the results, they offer important context for interpreting performance metrics and understanding the boundaries of the evaluation.

5 Results and Analysis

5.1 Overview and Evaluation Criteria

The evaluation of coreset methods focuses on their ability to balance clustering accuracy and computational efficiency across a wide range of datasets and parameter settings. Two primary performance metrics are used, as defined in the methodology: *clustering cost* and *total runtime*. For better clarity and presentation, both metrics were reported normalized relative to the *identity baseline* (the standard k-means algorithm applied to the full dataset using 10 k-means++ initializations) in all the plots.

For clustering cost, normalization allows the error introduced by each coreset to be measured relative to the full-data result. A normalized cost of **1.0** indicates that the coreset achieved the same clustering cost as the identity baseline, while higher values indicate some degree of approximation error. Likewise, runtime is reported as normalized total time, combining the time required for coreset construction and the k-means clustering step. A normalized runtime below 1.0 signifies a speed advantage compared to the baseline, while values above 1.0 indicate slower performance.

To quantify variability across repeated trials, error bars representing the *standard error* are shown on all plots. These are calculated as the standard deviation divided by the square root of the number of seeds (10), and provide a concise estimate of the uncertainty around the average reported values.

Given the large number of possible parameter combinations, especially within the grid-based methods, early exploratory analysis was used to determine representative settings for focused comparisons. One important decision was to **fix the coreset multiplier to** $m = 100$ for several parts of the analysis. This value was selected as it provided a clear midpoint: low enough to show the impact of reduced coreset sizes, but large enough to avoid excessive error or noise. It also strikes a balance between accuracy and performance that makes relative method differences easier to observe and interpret.

The remainder of the results and analysis proceeds in three stages. First, the Recursive Grid method is examined in detail, evaluating its different score functions and the effect of its start factor parameter. This is followed by a similar evaluation of the Guided Grid method. Finally, a head-to-head comparison is made between the best-performing configurations of all tested methods, including Grid Mean, Random Sampling, and the full-data baseline, to evaluate how well each method balances clustering fidelity with runtime efficiency.

5.2 Recursive Grid Evaluation

5.2.1 Score Function Comparison

To assess the influence of the scoring heuristic on Recursive Grid performance, five functions were tested with a fixed start factor of $s = 0.1$: *inertia*, *population*, *variance*, *MAD*, and *diameter*. These were evaluated over varying coreset sizes (m) and cluster counts (k), with metrics normalized against the identity baseline.

Across all values of m, each score function shows the expected trend of improved **clustering cost** with increased coreset size. As shown in the combined plot (Figure 1), *inertia* delivers the best cost performance overall, followed closely by *population*. The remaining functions: *variance*, *MAD*, and *diameter* perform slightly worse, with *MAD* clearly trailing behind at higher coreset sizes. Even so, the absolute spread between functions is small: a normalized cost gap of $\sim$0.02

separates the best and worst at $m = 50$, and the margin shrinks steadily with larger m.

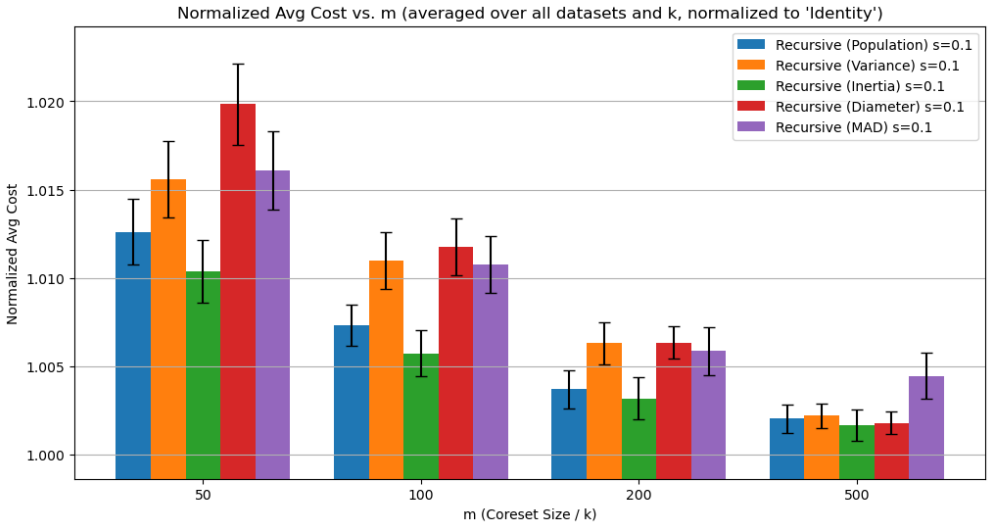


Figure 1: Normalized avg cost of Recursive score functions vs. m

These overall trends hold consistently across cluster counts as well. With $m = 100$ fixed, the combined cost vs. k plot (Figure 2) shows inertia maintaining its lead, followed by population. The cost tends to increase slightly from $k = 10$ but there is no clear pattern between $k = 20$ to $k = 50$. The differences remain minor, roughly 0.01 between the best and worst. Standard error is relatively higher at low k, especially for complex datasets like *US Census 2D* (Appendix A.1), which exhibits more erratic behavior and greater variability in its individual plot.

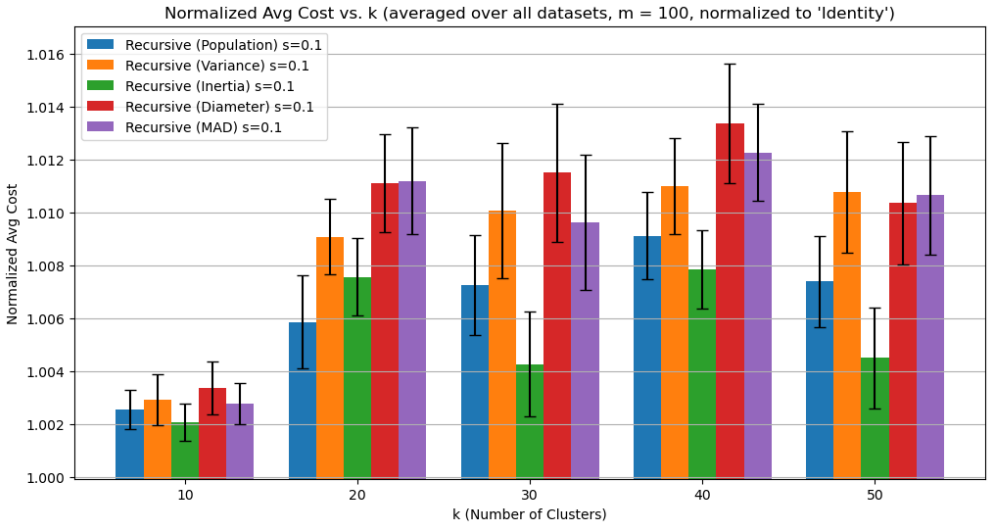


Figure 2: Normalized avg cost of Recursive score functions vs. k (for m=100)

Runtime comparisons show a roughly linear increase with m, with only modest variation between scoring functions. As shown in Figure 3, *variance* is marginally faster on average, but the differences are small and not likely impactful in practice. Individual plots confirm that variance tends to edge out others slightly in total time, but not enough to outweigh inertia's advantage in clustering accuracy.

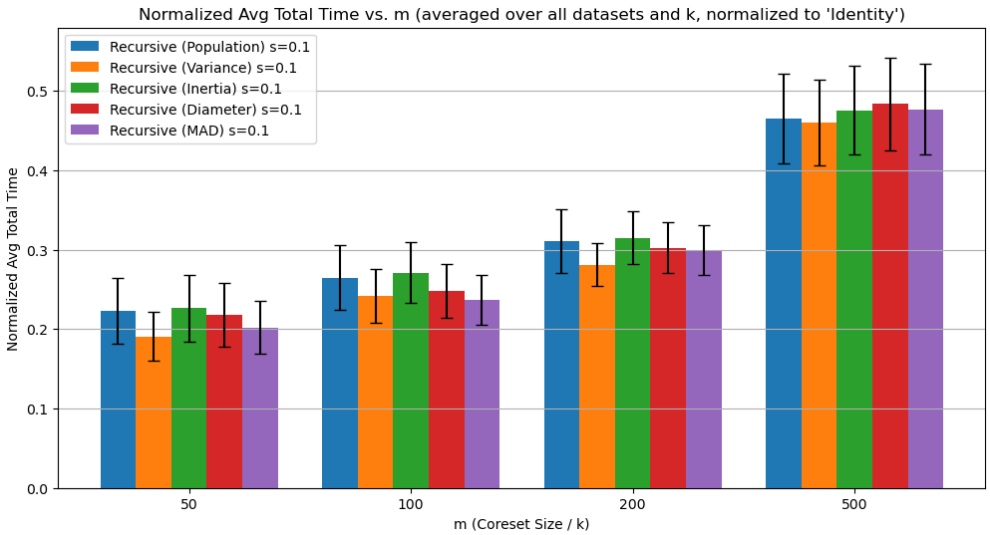


Figure 3: Normalized avg time of Recursive score functions vs. m

In the runtime vs. k plot (Figure 4), all functions show decreasing normalized time as k increases, a consequence of the baseline identity method taking longer with more clusters, while Recursive Grid's runtime remains largely unaffected by k. Again, function differences are small, with *variance* slightly ahead, and standard error remaining relatively large at low k. Dataset-level analysis from individual plots shows that *US Census 2D and 3D* (Appendix A.2) again exhibit the widest variation, while performance is steadier across the synthetic and Covertype datasets.

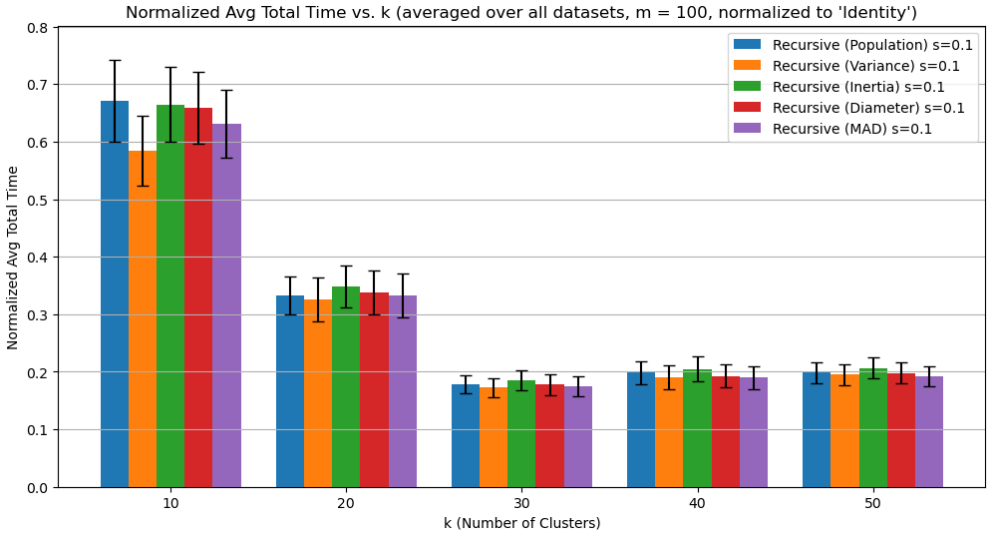


Figure 4: Normalized avg cost of Recursive score functions vs. k (for m=100)

Overall, *inertia* emerges as the best-performing score function, providing the lowest clustering cost with runtime comparable to the other options. *Population* is a close second, and might be preferable in some high-performance contexts due to its simpler calculation. However, given its consistent advantage in cost and broad stability across datasets, *inertia* was chosen as the default for all subsequent Recursive Grid evaluations.

5.2.2 Start Factor Analysis

To evaluate the effect of initial grid granularity, two start factors were compared using the inertia score: $s = 0.1$ (coarser grid, fewer starting cells) and $s = 0.9$ (finer grid, more starting cells). This parameter affects how the dataset is initially partitioned before recursive refinement begins, and can influence both clustering accuracy and runtime.

In terms of clustering cost, the coarser start factor ($s = 0.1$) consistently performs slightly better than the finer version, especially for smaller coreset sizes. As shown in Figure 5, the difference is most noticeable at $m = 50$, where the cost for $s = 0.9$ is $\sim$0.002 higher than for $s = 0.1$. This gap narrows as m increases, and at $m = 500$, the results converge almost completely, with $s = 0.1$ even being a bit higher than $s = 0.9$. It is possible that with a finer starting grid, more structure is captured early on, but fewer recursive refinements remain to adjust that structure, which is especially problematic for smaller m.

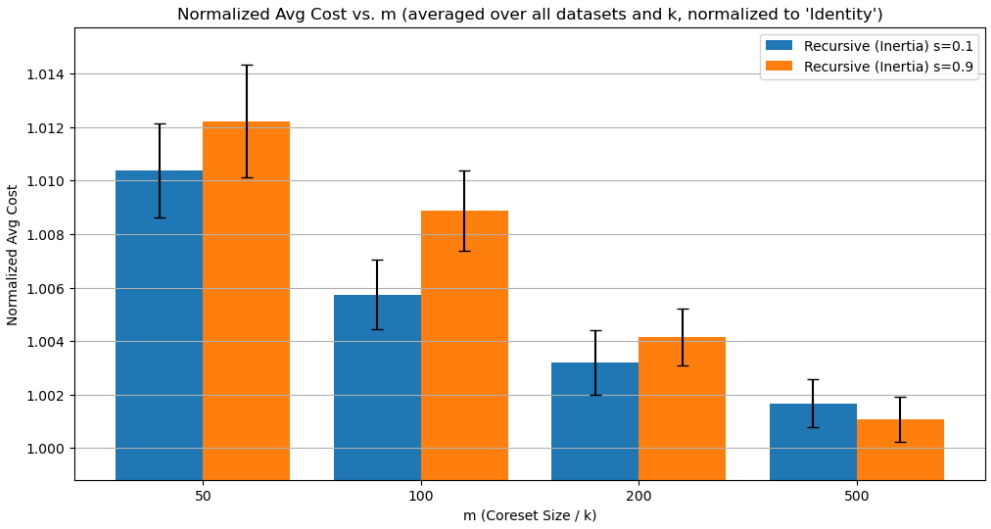


Figure 5: Normalized avg cost of Recursive start factors vs. m

The impact on runtime is more significant. As seen in Figure 6, the finer grid ($s = 0.9$) yields a clear speed advantage, with up to a **20% reduction** in total time at $m = 50$. This speedup diminishes as m grows, but remains present throughout. The benefit is intuitive: a finer initial grid produces more non-empty cells up front, requiring fewer recursive splits to reach the desired coreset size.

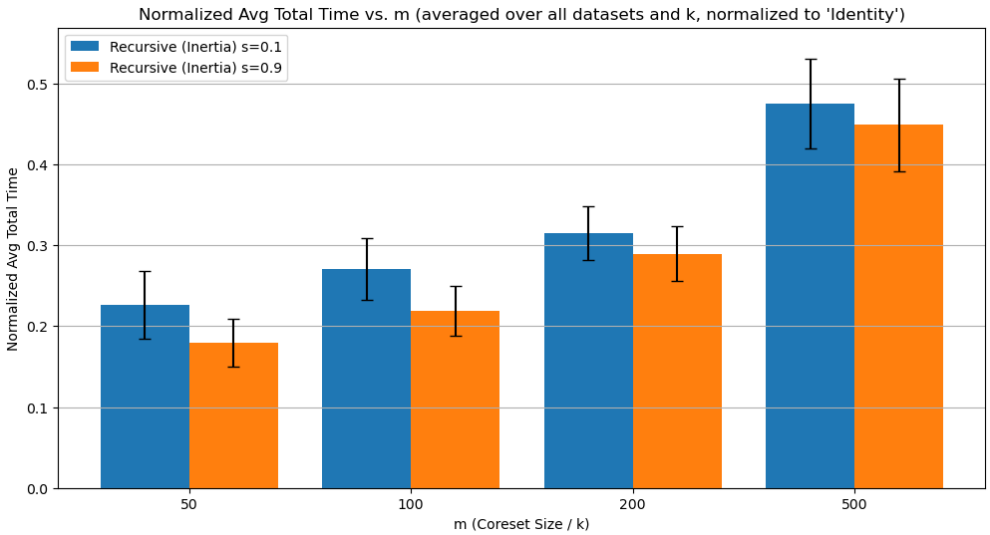


Figure 6: Normalized avg time of Recursive start factors vs. k (for m=100)

In summary, there is a modest trade-off between cost and runtime. Start factor $s = 0.1$ yields better clustering accuracy, particularly at low coreset sizes, while $s = 0.9$ delivers faster coreset construction. For this project, $s = 0.1$ was selected to prioritize accuracy and maintain consistency in comparisons with other methods.

5.3 Guided Grid Evaluation

5.3.1 Score Function Comparison

To evaluate the impact of different scoring heuristics in the Guided Grid method, performance was measured across the four cluster-aware scoring functions: *Error*, *Impurity*, *Label Entropy*, and *Margin*, all tested using a start factor of $s = 0.1$.

In terms of **clustering cost**, the *Error* score function consistently outperformed the others across the entire range of coreset sizes m. As shown in Figure 7, *Error* begins with a normalized cost close to 1.01 for $m = 50$, while the other heuristics show significantly higher values, with *Margin* reaching as high as 1.16. This trend holds across increasing coreset sizes, though the relative gaps narrow slightly at higher m. Figure 8 further confirms this dominance, with *Error* maintaining a consistently lower normalized cost across all values of k, while *Margin* continues to underperform. Notably, *Impurity* and *Label Entropy* often track closely together, forming a second tier of performance above *Margin* but below *Error*.

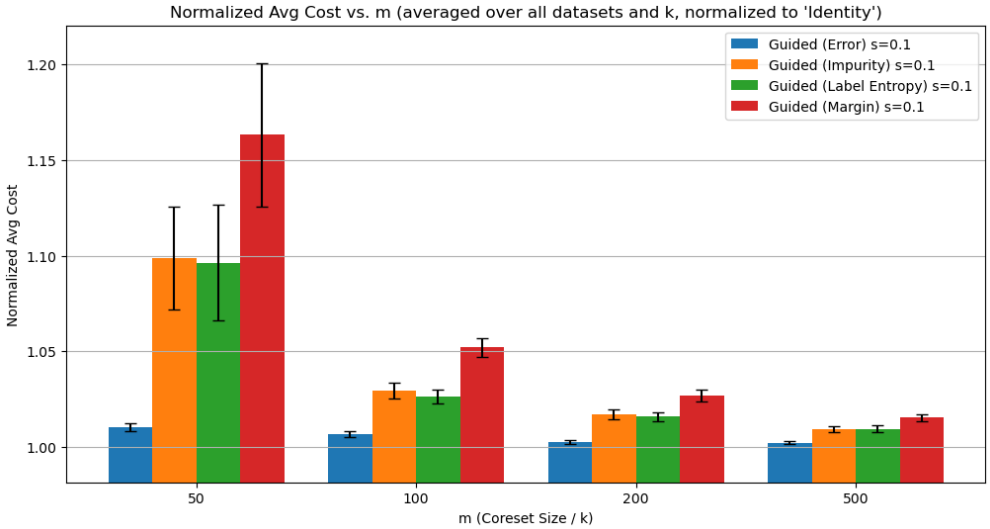


Figure 7: Normalized avg cost of Guided score functions vs. m

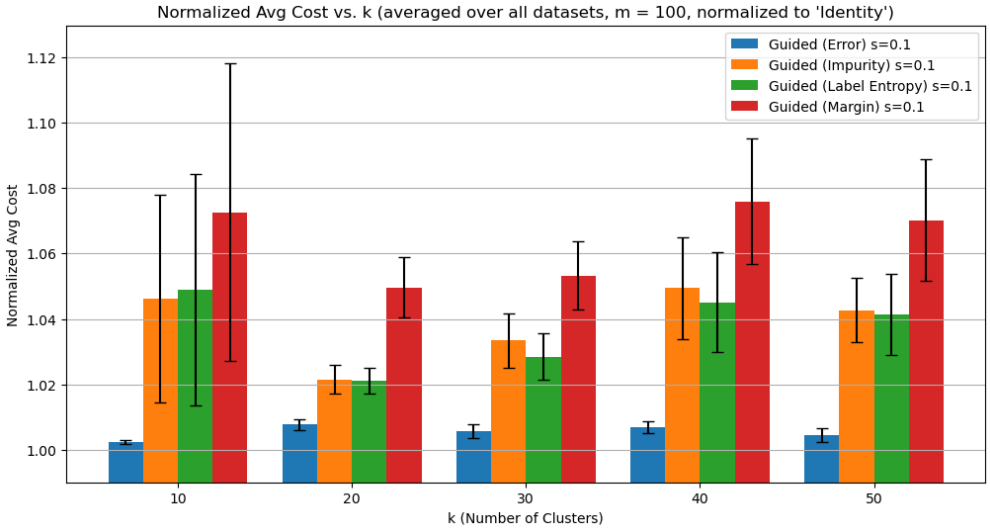


Figure 8: Normalized avg cost of Guided score functions vs. k (for m=100)

When examining individual datasets, *US Census 2D* (Appendix A.3) presents the most deviation from this trend. On that dataset, the *Error* score's performance fluctuated and even underperformed compared to *Impurity* at certain k values. However, this was an outlier. The general consistency of *Error* across the other datasets and configurations validates it as the most robust scoring heuristic.

In terms of **runtime**, the differences between *Error*, *Impurity*, and *Label Entropy* are relatively small and consistent across all tested values of m and k. As seen in Figures 9 and 10, all three scoring functions remain within roughly 1.3× of each other. In contrast, *Margin* incurs a much higher runtime, a consistent outlier in every configuration. This elevated overhead can be attributed to the computational structure of the *Margin* function, which involves sorting the squared distances to cluster centers for each point to calculate margin gaps.

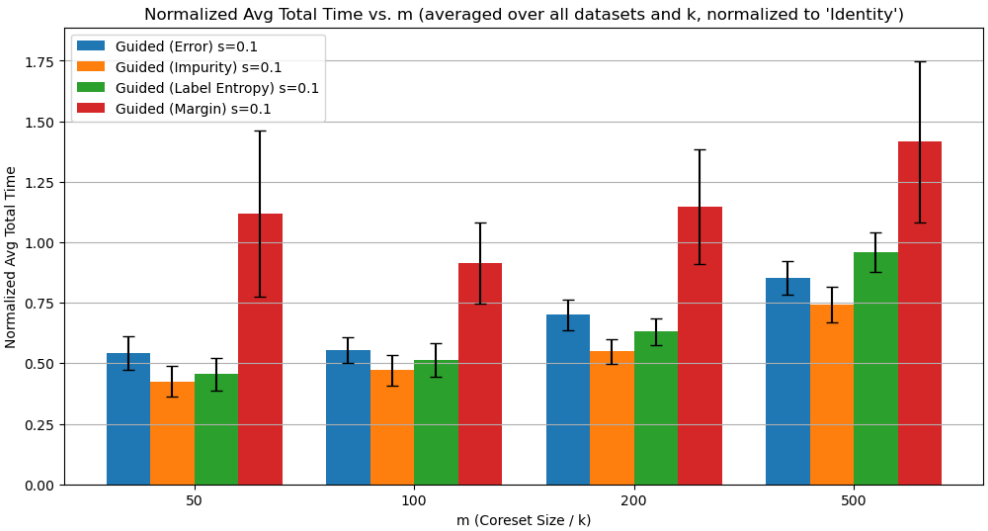


Figure 9: Normalized avg time of Guided score functions vs. m

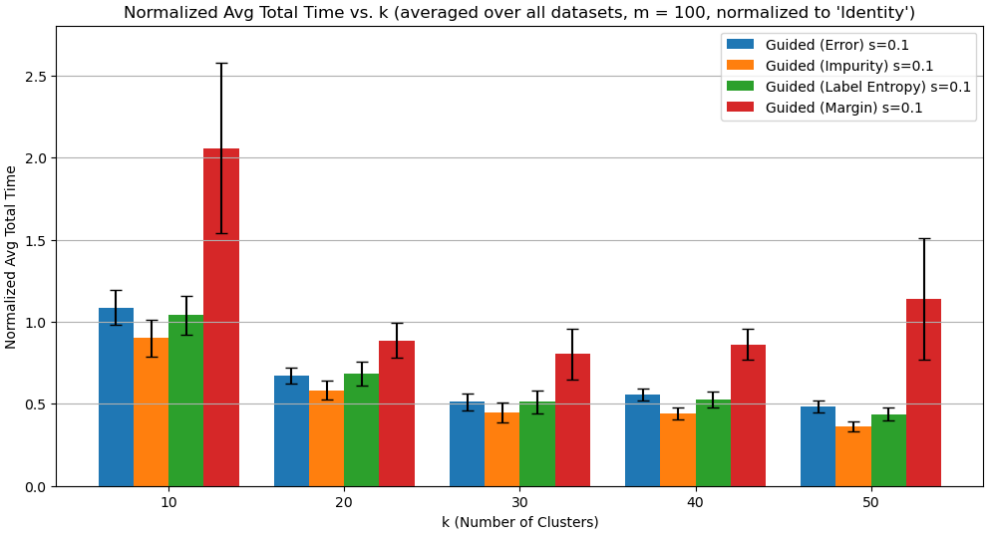


Figure 10: Normalized avg time of Guided score functions vs. k (for m=100)

Taken together, the *Error* score was selected as the best-performing heuristic for Guided Grid, offering the best balance between cost and runtime. While not the absolute fastest, its cost improvement was significant and consistent, and its runtime remained competitive, only marginally slower than the fastest (*Impurity*) but drastically better in terms of clustering accuracy.

5.3.2 Start Factor Analysis

After selecting the *Error* score function as the best-performing heuristic, further analysis was conducted on the impact of the **start factor** parameter (s), which controls the granularity of the initial grid. A higher start factor ($s = 0.9$) results in a finer initial grid, producing more non-empty cells, while a lower start factor ($s = 0.1$) yields a coarser grid with fewer initial cells.

The clustering cost comparison is shown in Figures 11 and 12. Overall, $s = 0.1$ slightly outperforms $s = 0.9$ in terms of accuracy, especially at lower coreset sizes. For $m = 50$, $s = 0.1$ achieves a normalized cost of approximately 1.010, whereas $s = 0.9$ reaches closer to 1.013, a modest but noticeable difference. However, as the coreset size increases, the accuracy gap narrows. By $m = 500$, the normalized costs of both configurations converge to nearly the same value. The difference across k values follows a similar pattern but in reverse, with the gap between both increasing as k increases. The gap between $s = 0.1$ and $s = 0.9$ remains quite small,, typically less than 0.003, and comparable to the margin of standard error.

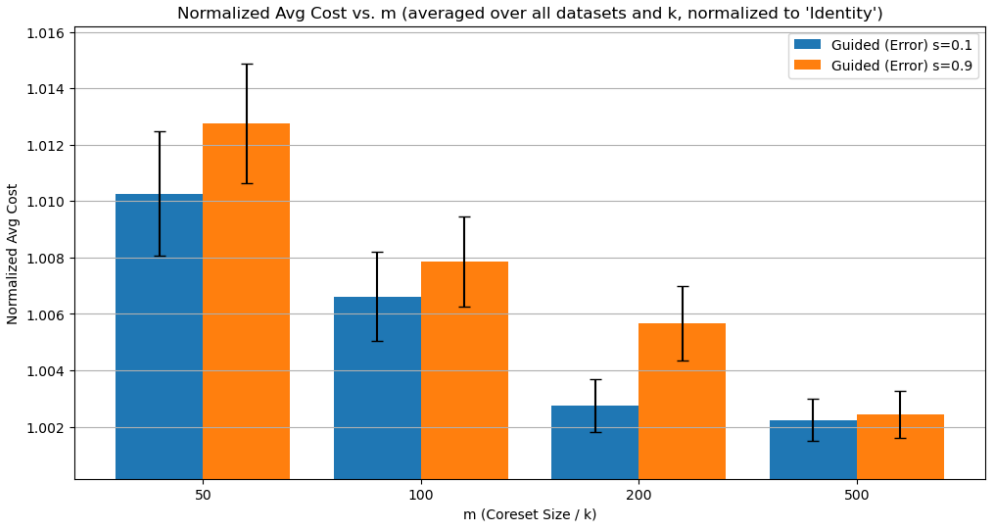


Figure 11: Normalized avg cost of Guided start factors vs. m

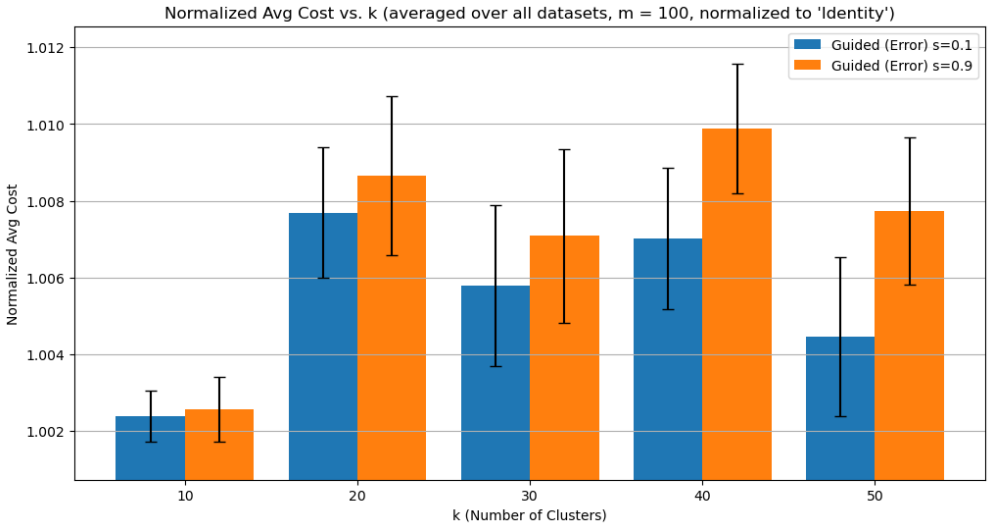


Figure 12: Normalized avg cost of Guided score factors vs. k (for m=100)

In contrast, **runtime** differences between the two start factors are much more pronounced. As seen in Figure 13, the finer grid ($s = 0.9$) results in significantly lower total time. For example, at $m = 50$, the total runtime is roughly halved, and although this speed-up diminishes at larger coreset sizes, $s = 0.9$ still maintains a clear advantage across all m. The trend holds across k as well (Figure 14), with $s = 0.9$ cutting total runtime by nearly 50% at higher k values.

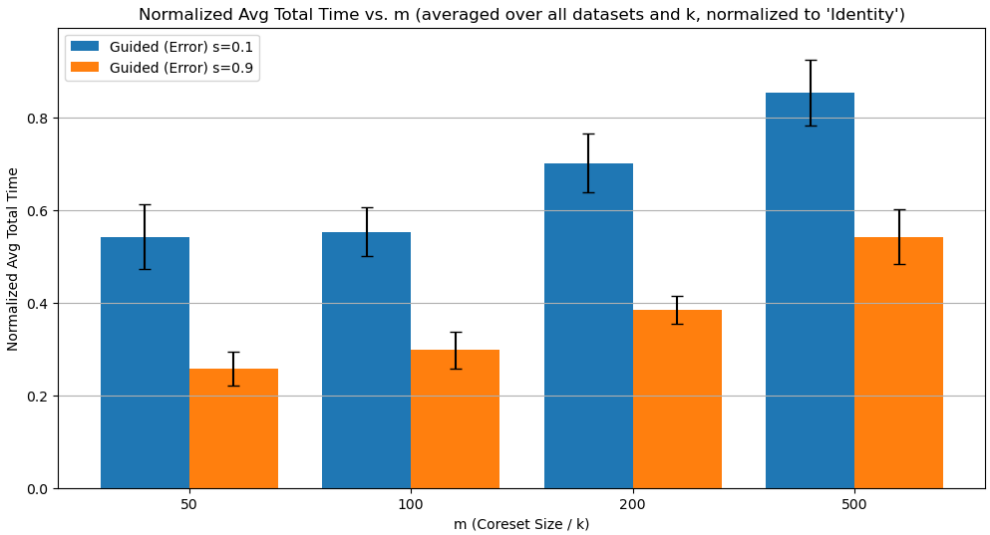


Figure 13: Normalized avg time of Guided start factors vs. m

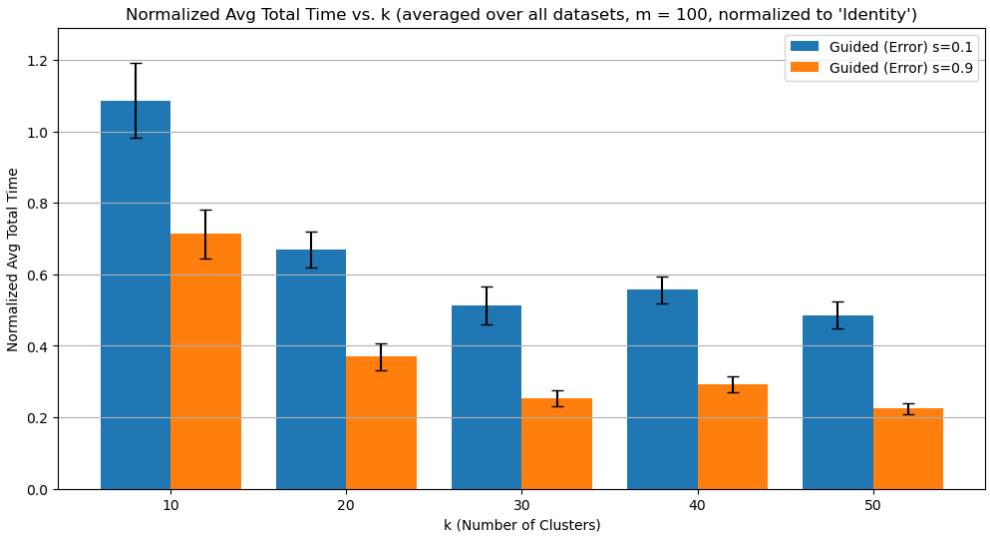


Figure 14: Normalized avg time of Guided start factors vs. k (for m=100)

This performance gap is due to the number of splits and k-means runs required to reach the target coreset size. Starting with a finer grid (higher s) means that fewer operations are needed later on, which significantly reduces computational overhead. The coarser starting point of $s = 0.1$ requires more operations to achieve the same granularity, increasing both time and complexity.

In conclusion, although $s = 0.1$ offers slightly better accuracy, the substantial runtime advantage of $s = 0.9$ makes it the preferred setting for most practical scenarios. The trade-off is well-justified: cost differences remain minor and often fall within the range of standard error, while the speedup is consistent and substantial across all tested configurations. For this reason, $s = 0.9$ was selected as the final start factor used in Guided Grid's full comparison.

5.4 Full Comparison Across Methods

With the best configurations selected for each grid-based method, this final comparison evaluates how they perform against each other and against two baselines: full-data *Identity k-means* and *Random Sampling*. Recursive Grid is tested using the *Inertia* score function and a coarse start factor ($s = 0.1$), chosen for its consistently low clustering cost. Guided Grid uses the *Error* score function with a finer start factor ($s = 0.9$), prioritising faster runtimes with only minor cost trade-offs. Grid Mean remains fixed with a grid size selected to target the appropriate coreset size. Random Sampling is included for completeness, but its much higher error means it is excluded from most *cost* plots for readability. Additionally, *Identity* was not included in the **vs. m** plots due to it not having a set size (as it uses the full data), the overall results are still normalized to *Identity* on the full data which still allows for comparison.

5.4.1 Cost Comparison

Across all methods, increasing the coreset size m improves clustering cost as expected, with the gains most noticeable at lower sizes. In the combined plot of **cost vs.** m (Figure 15), Recursive Grid clearly performs the best overall, maintaining a consistent edge over Guided and Grid Mean across the entire range. The difference is most noticeable at smaller sizes like $m = 100$ and $m = 200$, where Recursive leads by approximately 0.0025 in normalized cost. At $m = 500$, the gap narrows significantly, and all three grid-based methods begin to converge, indicating that with enough resolution, even simpler strategies like Grid Mean can approach the accuracy of more adaptive techniques.

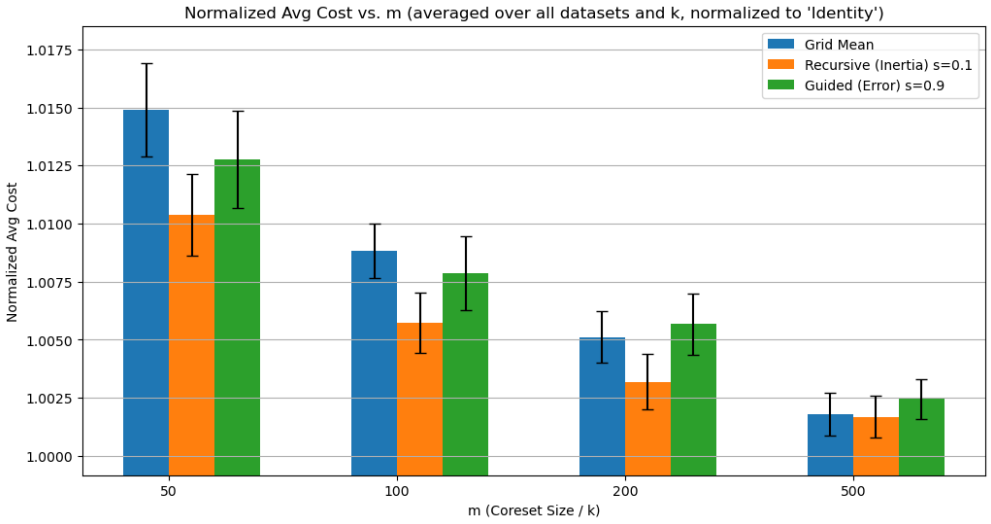


Figure 15: Normalized avg cost of all methods (excluding random) vs. m

Grid Mean, which starts off with the weakest performance at small m, improves steadily as more representative points are included. Interestingly, it slightly outperforms Guided Grid at $m = 500$, suggesting that while adaptivity offers benefits at tighter coreset sizes, it becomes less crucial as coresets grow larger. Guided Grid remains stable throughout, but rarely surpasses Recursive in terms of cost.

When shifting focus to **cost vs.** k (Figure 16), the relative rankings between methods remain consistent. Recursive again takes the lead across almost all cluster counts, with Grid Mean and Guided Grid alternating second and third depending on the specific value of k. Grid Mean

performs particularly well at lower k values (10 and 20), while Guided Grid gains a small edge at higher k (30 and 50). The differences are small, no more than about 0.002 in normalized cost, but consistent. A notable trend is the lower overall cost observed at $k = 10$, where the clustering task is simpler and more stable. Beyond that, normalized costs tend to plateau around 1.01 for all methods.

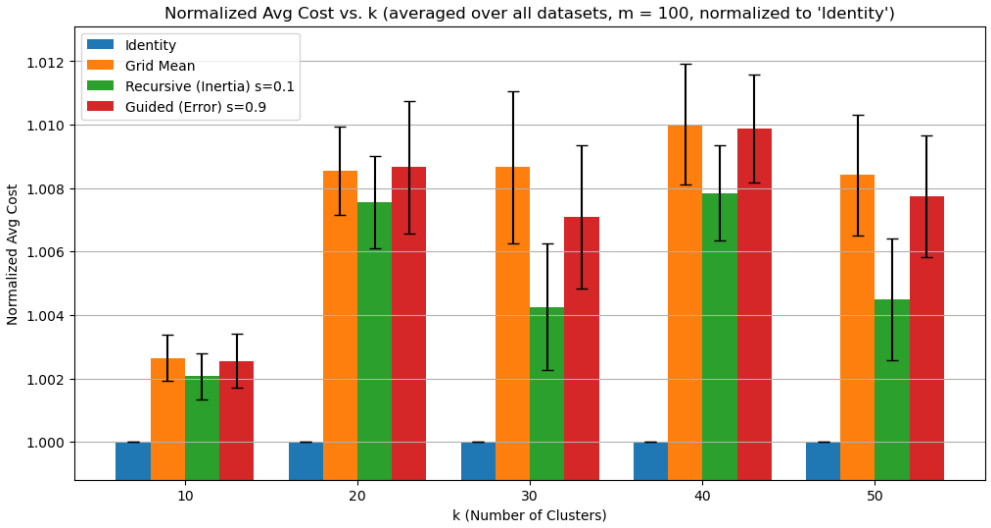


Figure 16: Normalized avg cost of all methods (excluding random) vs. k (for m=100)

In a few individual cases, particularly for the US Census datasets (Appendix A.4), Recursive Grid even manages to slightly outperform the Identity baseline, for instance, at $k = 30$ in US Census 2D, and $k = 50$ in 3D. These edge cases suggest that grid-based summarisation might, in some situations, smooth out noise or overfitting that full-data k-means may be sensitive to, especially after PCA compression.

Random Sampling, shown in Figure 17, clearly lags behind the other methods in accuracy. At $m = 50$, its average normalized cost is above 1.22, indicating an error rate exceeding 22%. This drops to around 1.075 at $m = 100$, and down to 1.025 by $m = 500$, but even then, it fails to match the performance of any grid-based strategy. Its high variance at low coreset sizes also makes it unreliable, reinforcing its role as a cost upper bound.

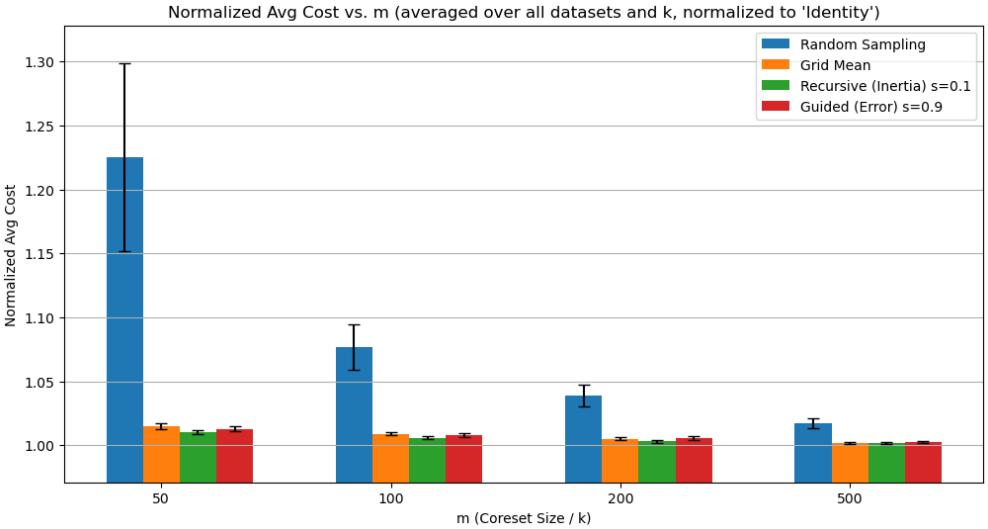


Figure 17: Normalized avg time of all methods vs. m

5.4.2 Time Comparison

Runtime comparisons tell a different story. As shown in the **time vs.** m plot (Figure 18), Random Sampling is easily the fastest method overall, completing coreset construction almost instantly at smaller m. However, its runtime grows more steeply than the others, reaching 0.23 normalized time by $m = 500$. This suggests that while it is fast, it does not scale quite as gracefully as expected, possibly due to increased sampling and data handling overhead as coreset sizes grow.

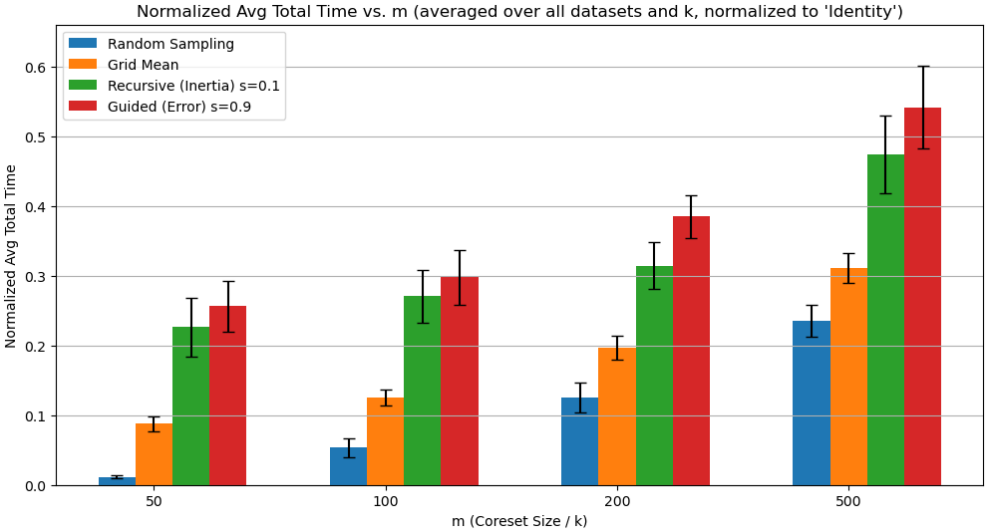


Figure 18: Normalized avg time of all methods vs. m

Grid Mean stands out as the fastest among the grid-based methods, remaining well below the others at every tested m. Its simplicity, relying only on a single pass through the data to compute means, gives it a consistent speed advantage. Recursive Grid and Guided Grid both take longer, but follow similar trends, increasing linearly with coreset size. The gap between them stays

fairly stable: Guided is generally about 0.15 normalized time units slower than Recursive. This offset is likely due to the repeated k-means runs inside the Guided method, as well as its more complex scoring functions.

In the **time vs.** k plot (Figure 19), another interesting trend appears. Recursive and Guided both become faster (in relative terms) as k increases, most notably from $k = 10$ to $k = 30$. This drop is driven by the fact that baseline k-means (*Identity*) becomes more expensive as k increases, while coreset construction remains relatively unaffected. Grid Mean and Random Sampling, by contrast, show flat trends across k, indicating that their runtime is largely independent of the number of clusters.

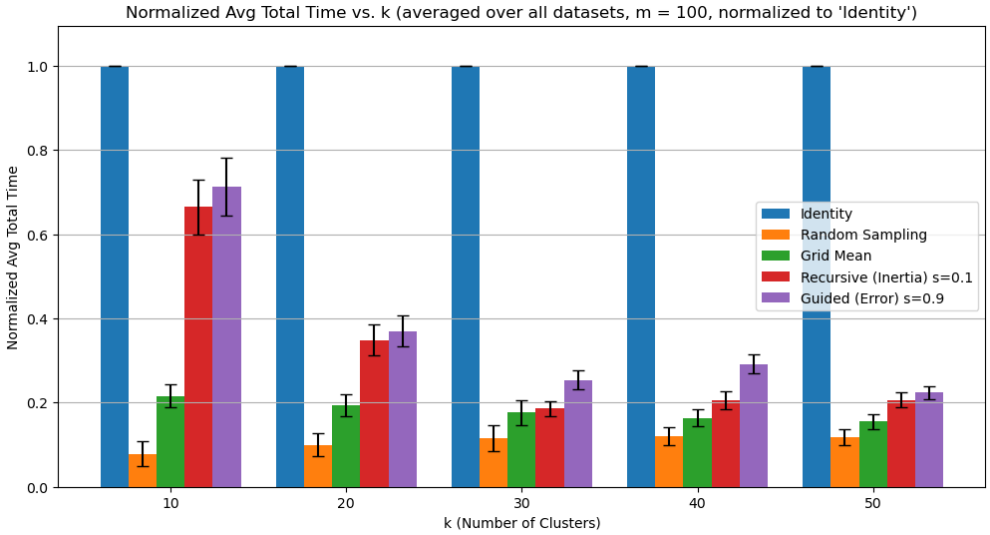


Figure 19: Normalized avg time of all methods vs. k (for m=100)

The dataset-specific plots further highlight performance variation (Appendix A.5). In Covertype 2D and 3D, Recursive and Guided perform particularly well in terms of runtime, with Guided even outperforming the other methods at $k = 50$. However, on the US Census datasets, their runtimes spike significantly. Recursive reaches close to 0.95 normalized time in US Census 2D, while Guided goes beyond 1.1 in US Census 3D. These differences reflect how spatial distribution and data volume post-PCA influence the cost of adaptive splitting and scoring.

5.4.3 Summary of Trade-offs

Bringing both cost and runtime together, a few clear patterns emerge:

- **Recursive Grid** delivers the best clustering cost across nearly all test settings, often outperforming the baselines and staying highly consistent.
- **Grid Mean** is the most efficient grid-based method, and with large coresets, its cost approaches that of Recursive, making it a strong choice when speed is critical.
- **Guided Grid**, while slower, generally outperforms Grid Mean at lower coreset sizes and higher cluster counts, but its added complexity does not always justify the marginal gains.
- **Random Sampling**, although extremely fast, suffers from poor and highly variable clustering accuracy, especially at small m. It is best interpreted as a lower-bound benchmark rather than a practical alternative.

Overall, **Recursive Grid** offers the most favourable **cost-runtime trade-off**, particularly when high accuracy is required, while **Grid Mean** excels in **low-latency scenarios**. The choice between them ultimately depends on the application's tolerance for error and constraints on processing time.

5.5 Discussion

The final comparison reveals several important insights into how the different coreset methods behave under varying conditions. Most notably, *Recursive Grid* consistently emerges as the top performer in terms of clustering cost, delivering the most faithful approximation of full-data k-means across datasets, cluster counts, and coreset sizes. This performance reflects its ability to focus resolution adaptively, using scoring functions like *Inertia* to identify and refine complex regions without overcommitting to areas that offer little gain. While it does not outperform others in raw speed, its runtime remains reasonable, and its cost savings make it an attractive choice for tasks where clustering quality matters.

In contrast, **Guided Grid**, which was designed as an extension of Recursive Grid, does not meet expectations in the final results. While its use of k-means feedback seems promising in theory, it lags behind Recursive in both cost and time for most scenarios. This may be due to the nature of its scoring heuristics, which rely mainly on point-to-cluster relationships, failing to fully capture local structure within cells. Recursive's more direct analysis of spatial spread and distribution allows it to make smarter splitting decisions without requiring expensive feedback loops. It's possible that a more hybrid approach, combining elements of both scoring strategies, could yield better results, for example, integrating measures of spread (like inertia or variance) into the cluster-aware scoring of Guided Grid.

Grid Mean, while significantly simpler, proves surprisingly effective, especially at larger coreset sizes. Although it lacks any adaptivity, its uniform treatment of space and use of weighted centroids still allows it to capture global structure well. By $m = 500$, it nearly matches Recursive Grid in accuracy and even surpasses Guided in several configurations. Combined with its extremely fast runtime, this makes it a compelling option in scenarios where quick approximation is preferable to fine-tuned adaptivity. However, its performance drops sharply at small coreset sizes, confirming that without refinement, grid summarisation struggles to represent finer clustering boundaries effectively.

The practical implications of these findings are clear. If runtime is a critical bottleneck, for example, in real-time applications or large batch processing, **Grid Mean** offers a robust and efficient solution, especially when paired with moderate to large coresets. If clustering accuracy is paramount and time constraints are more relaxed, **Recursive Grid** remains the most capable option. **Guided Grid**, while not outperforming either baseline, may still have value in contexts where k-means cluster structure provides important cues for refinement, though its current scoring design would benefit from further refinement.

Interestingly, the final results diverge somewhat from the initial expectations based on **preliminary experiments**. In the early tests, the *Population* and *Variance* score functions appeared most promising for Recursive Grid, and *Impurity* for Guided Grid. However, the full-scale evaluation, which spans multiple datasets, cluster counts, and coreset sizes, revealed that *Inertia* consistently outperforms its counterparts for Recursive, while *Error* performs best for Guided. This highlights the importance of broad-based testing. Tuning parameters on a limited slice of the configuration space can obscure trends that only emerge at scale. It also reinforces that coreset behavior depends heavily on dataset characteristics, further justifying the use of both synthetic and real-world data in the benchmark.

Taking a step back, the central question of this project was:

can grid-based partitioning coresets provide a meaningful trade-off between clustering accuracy and computational efficiency for k-means clustering in low dimensions?

Based on the results, the answer is a resounding **Yes**. Grid-based coresets, particularly Recursive Grid, consistently achieve a favorable balance between runtime and clustering quality. Even Grid Mean, despite its simplicity, provides strong performance in many practical scenarios. While some methods (notably Guided Grid) fell short of their conceptual potential, the overall results show that spatial partitioning is a powerful and viable approach to scalable clustering, especially in low-dimensional settings where grid representations remain efficient.

5.6 Limitations and Future Testing Extensions

Although the evaluation results strongly support the effectiveness of grid-based coresets for k-means clustering, several experimental constraints limited the breadth and depth of the conclusions. These limitations, while not undermining the core findings, point to valuable opportunities for future improvement.

One key limitation lies in the implementation and runtime environment. The grid-based methods were implemented in Python, whereas the baseline k-means used scikit-learn's highly optimized C++ backend. This mismatch introduces a bias in runtime comparisons, as Python's interpreted nature and memory handling overhead can distort performance measurements, particularly for Recursive and Guided Grid, which involve recursive splitting and frequent data manipulation. Consequently, their true efficiency relative to the k-means baseline may be underestimated. Reimplementing the grid-based methods in a compiled language like C++, or accelerating them with just-in-time tools such as Numba or Cython, would provide more realistic benchmarks and support fairer comparisons.

Another constraint was the restricted range of coreset sizes. Experiments were limited to just four m values: 50, 100, 200, and 500. While this range was sufficient to identify broad trends, it likely missed more subtle behaviors, such as diminishing returns or accuracy saturation, that might appear at higher scales. Expanding the range of m values in future work, particularly toward larger coresets, could help capture these longer-term trends and refine understanding of each method's scalability.

In addition, the limited number of trials introduced some variability in the results. Each experiment was repeated across 10 random seeds, which was enough to highlight general patterns but not sufficient to fully stabilize metrics like normalized cost, especially for more variable methods such as Random Sampling. Increasing the number of trials in future experiments would help tighten error margins and improve confidence in the observed differences, especially for datasets with higher noise or sensitivity to initialization.

The parameter space explored for Recursive and Guided Grid was also relatively narrow. Only two start factors ($s = 0.1$ and $s = 0.9$) were tested, and Guided Grid was run with a fixed `km_interval` and default k-means settings. While these constraints kept the experiment tractable, they also left open the possibility that intermediate parameter values or alternative configurations might yield better results. A more exhaustive sweep, exploring a finer grid of start factors, varying cluster feedback frequency, or adjusting internal k-means iterations, could uncover more optimal settings and provide a clearer picture of the tuning landscape.

Another important omission was the lack of comparison to other coreset benchmarks. The evaluation focused entirely on the newly proposed grid-based methods, without including more established techniques such as BICO [7] or Lightweight Coreset [4]. These algorithms are well-known in the literature and serve as strong baselines for both cost and runtime performance. Integrating them into future evaluations would position grid-based methods within the broader

k-means acceleration space, offering more comprehensive insights into their relative strengths and weaknesses.

Finally, there is a structural limitation in how scoring heuristics are applied, particularly in Guided Grid. While Recursive Grid uses purely geometric information for splitting, Guided Grid relies on cluster-aware metrics that incorporate feedback from interim k-means results. This distinction means Guided Grid may miss opportunities to refine areas with high intra-cell variance if those areas don't align with current cluster assignments. A promising extension would involve hybrid scoring strategies that blend local geometric indicators (e.g., inertia or variance) with global cluster-aware heuristics like margin or impurity. Such an approach could improve refinement precision and enhance cost-runtime trade-offs, particularly in more complex or noisy datasets.

In summary, while the core conclusions of this study remain well-supported, these limitations identify concrete avenues for expanding, refining, and validating the results in future work.

6 Conclusion

This project set out to examine whether **grid-based partitioning coresets** can offer an effective balance between **computational efficiency** and **clustering accuracy** for large-scale k-means clustering in low-dimensional data. Through the design and evaluation of three coreset strategies: Grid Mean, Recursive Grid, and Guided Grid, and a comprehensive experimental pipeline, this goal was explored across a range of datasets, cluster sizes, and coreset configurations.

The findings were encouraging. Recursive Grid consistently achieved the best clustering cost, demonstrating that spatial adaptivity guided by scoring heuristics can yield compact, representative coresets. Grid Mean, though much simpler, offered the fastest runtimes and was often competitive in accuracy at higher coreset sizes. Guided Grid, while conceptually promising, showed that more complexity does not always translate into better outcomes, its feedback loop introduced overhead without significantly improving cost.

Across all methods, the results confirmed that grid-based approaches can indeed produce coresets that closely approximate full-data k-means results while reducing overall runtime. The gains in efficiency were especially notable given the implementation constraints, the methods were written in Python, whereas the baseline k-means used a highly optimized C++ backend. This makes the observed time savings all the more meaningful.

There were, of course, limitations. The restricted parameter ranges, modest number of seeds, and differences in runtime environments highlight areas where further testing would add confidence and nuance. Yet even within these constraints, clear patterns emerged that support the utility of grid-based coresets for practical clustering tasks.

In answering the core research question: *Can grid-based partitioning coresets provide a meaningful trade-off between clustering accuracy and runtime for k-means clustering in low dimensions?* the results provide a confident **Yes**. These methods offer scalable, interpretable, and surprisingly effective strategies for summarizing large datasets with minimal loss in clustering quality.

Future work may explore improved score functions, tighter integration of geometric and cluster-aware signals, or reimplementation in high-performance environments. Nevertheless, this project demonstrates that with careful design, grid-based coresets can serve as **practical and effective tools** for accelerating k-means clustering, especially in low-dimensional settings where spatial structure can be efficiently exploited.

References

[1] Marcel R. Ackermann, Marcus Märtens, Christoph Raupach, Karsten Swierkot, Christiane Lammersen, and Christian Sohler. Streamkm++: A clustering algorithm for data streams. In *Proceedings of the 12th International Conference on Algorithmic Learning Theory (ALT)*, pages 173–187, 2010.

[2] Rakesh Agrawal, Johannes Gehrke, Dimitrios Gunopulos, and Prabhakar Raghavan. Automatic subspace clustering of high dimensional data for data mining applications. In *Proceedings of the 1998 ACM SIGMOD International Conference on Management of Data*, pages 94–105, 1998.

[3] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1027–1035, 2007.

[4] Olivier Bachem, Mario Lucic, Hamed Hassani, and Andreas Krause. Scalable k-means clustering via lightweight coresets. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery Data Mining (KDD)*, pages 1116–1125, 2018.

[5] Dick J. Brus. Spatial sampling with r, 2023. Accessed: 2025-07-19.

[6] Marco Capó, Antonio Pérez, and José A. Lozano. An efficient k-means clustering algorithm for massive data. *Knowledge-Based Systems*, 117:56–69, 2017.

[7] Sara Ceschia, Aristides Gionis, Tobias Müller, and Thomas Seidl. Bico: Birch meets coresets for k-means clustering. In *Proceedings of the 15th International Conference on Extending Database Technology (EDBT)*, pages 435–446, 2011.

[8] Vincent Cohen-Addad, David Saulpic, and Chris Schwiegelshohn. A new coreset framework for clustering. In *Proceedings of the 60th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 396–409, 2019.

[9] Philip M. Dixon. *Nearest-Neighbor Methods*. Wiley, 2002.

[10] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231, 1996.

[11] Dan Feldman and Michael Langberg. A unified framework for approximating and clustering data. *Journal of the ACM*, 63(2):1–35, 2016.

[12] Dan Feldman, Melanie Schmidt, and Christian Sohler. Turning big data into tiny data: Constant-size coresets for k-means, pca, and projective clustering. In *Proceedings of the 24th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1434–1453, 2013.

[13] Maximilian Fiedler, Peter Gritzmann, and Fabian Klemm. On resolution coresets for constrained clustering. *Journal of Computational Mathematics and Data Science*, 12:100100, 2024.

[14] Gereon Frahling and Christian Sohler. Coresets in dynamic geometric data streams. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC)*, pages 209–217, 2005.

[15] Gereon Frahling and Christian Sohler. A fast k-means implementation using coresets. *International Journal of Computational Geometry Applications*, 18(06):605–625, 2008.

[16] Sariel Har-Peled and Sumiran Mazumdar. On coresets for k-means and k-median clustering. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing (STOC)*, pages 291–300, 2004.

[17] Tapas Kanungo, David M. Mount, Nathan S. Netanyahu, Christine D. Piatko, Ruth Silverman, and Angela Y. Wu. An efficient k-means clustering algorithm: Analysis and implementation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(7):881–892, 2002.

[18] Stuart P. Lloyd. Least squares quantization in pcm. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[19] J. MacQueen. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1:281–297, 1967.

[20] NumPy Developers. Numpy: Fundamental package for scientific computing in python. `https://numpy.org`. Accessed 2025.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in python. `https://scikit-learn.org`, 2011. Accessed 2025.

[22] Jeff M. Phillips. Coresets and sketches. *arXiv preprint arXiv:1601.00617*, 2016.

[23] Hanan Samet. The quadtree and related hierarchical data structures. *ACM Computing Surveys*, 16(2):187–260, 1984.

[24] Chris Schwiegelshohn and Omar Ali Sheikh-Omar. An Empirical Evaluation of k-Means Coresets. In Shiri Chechik, Gonzalo Navarro, Eva Rotenberg, and Grzegorz Herman, editors, *30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 84:1–84:17, Dagstuhl, Germany, 2022. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[25] Wei Wang, Jiong Yang, and Richard Muntz. Sting: A statistical information grid approach to spatial data mining. *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*, pages 186–195, 1997.

A Additional Figures

All Plots as well as the results in csv form have been included in the same folder as the code, they have not all been included here due to the significant volume of total plots (over 100) and data (over 2500 lines). Below includes some plots showing outliers that are referred to in the report.

A.1 Recursive Results Cost Outliers

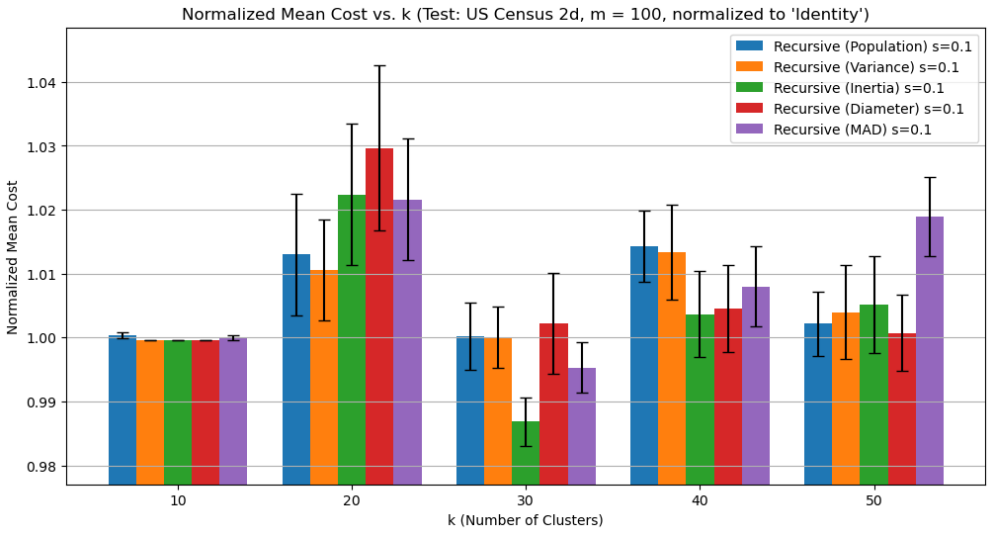


Figure 20: Normalized mean cost of Recursive score functions vs k (US Census 2d, m=100)

A.2 Recursive Results Time Outliers

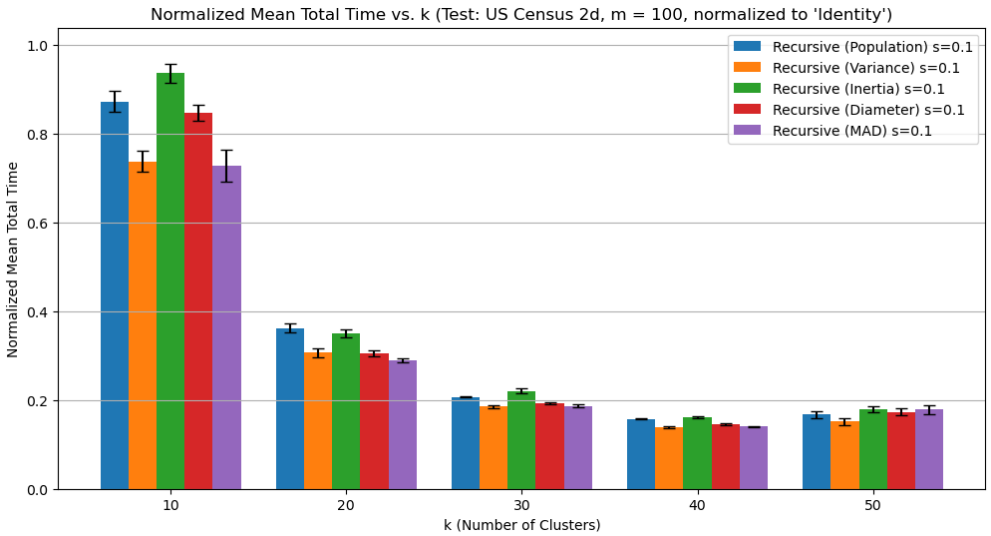


Figure 21: Normalized mean time of Recursive score functions vs k (US Census 2d, m=100)

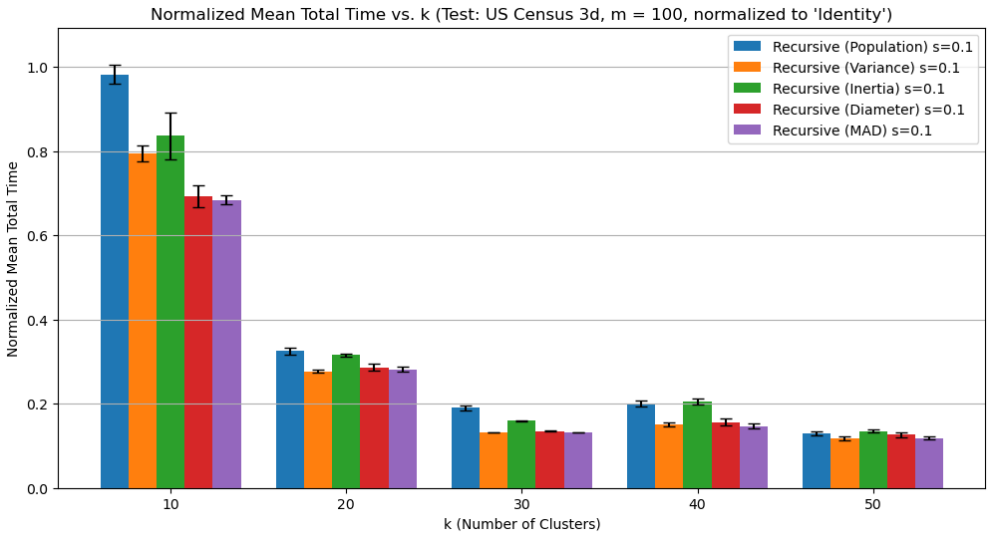


Figure 22: Normalized mean time of Recursive score functions vs k (US Census 3d, m=100)

A.3 Guided Results Time Outliers

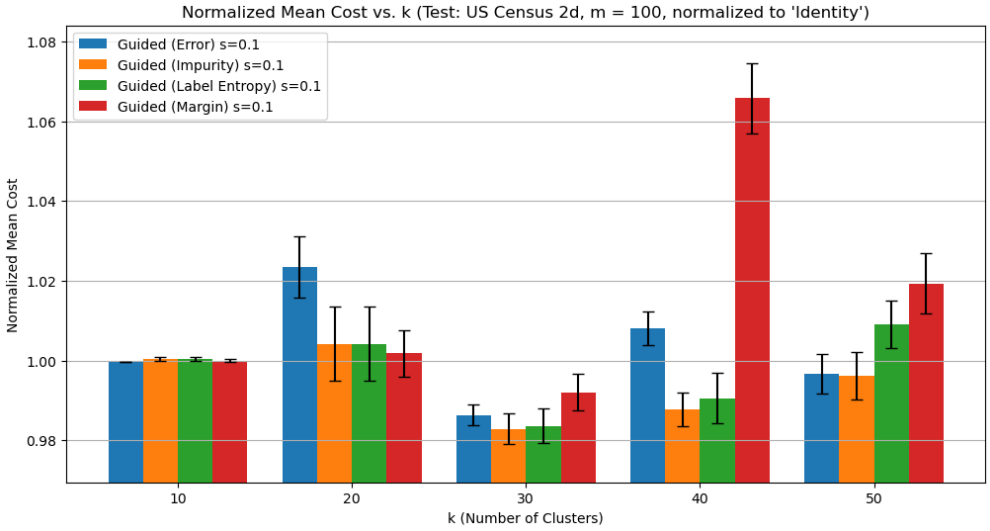


Figure 23: Normalized mean cost of guided score functions vs k (US Census 2d, m=100)

A.4 Full Results Cost Outliers

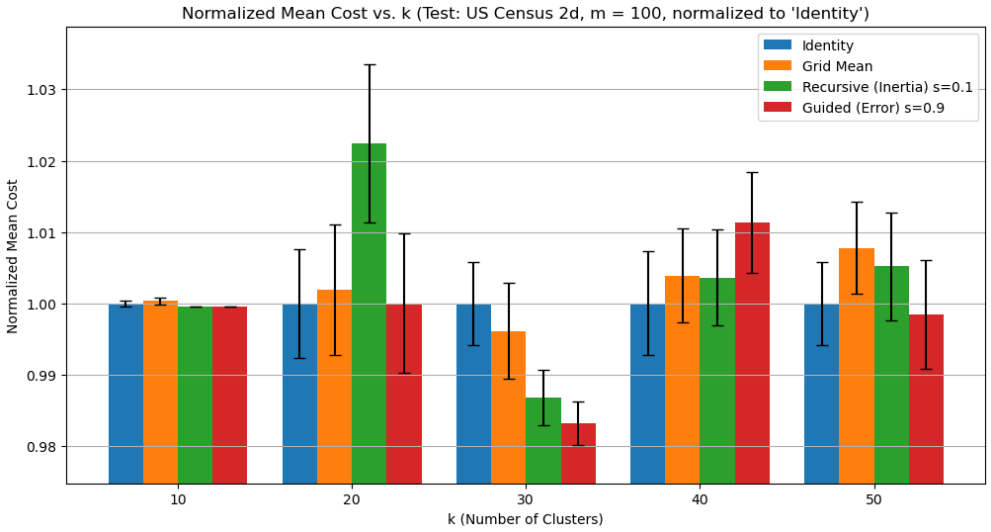


Figure 24: Normalized mean cost of all methods (excluding random) vs k (US Census 2d, m=100)

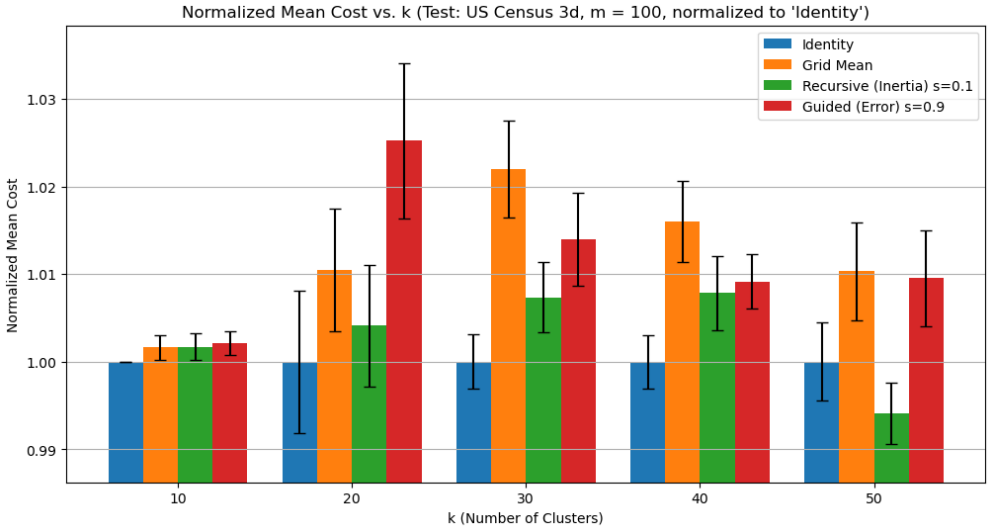


Figure 25: Normalized mean cost of all methods (excluding random) vs k (US Census 3d, m=100)

A.5 Full Results Time Outliers

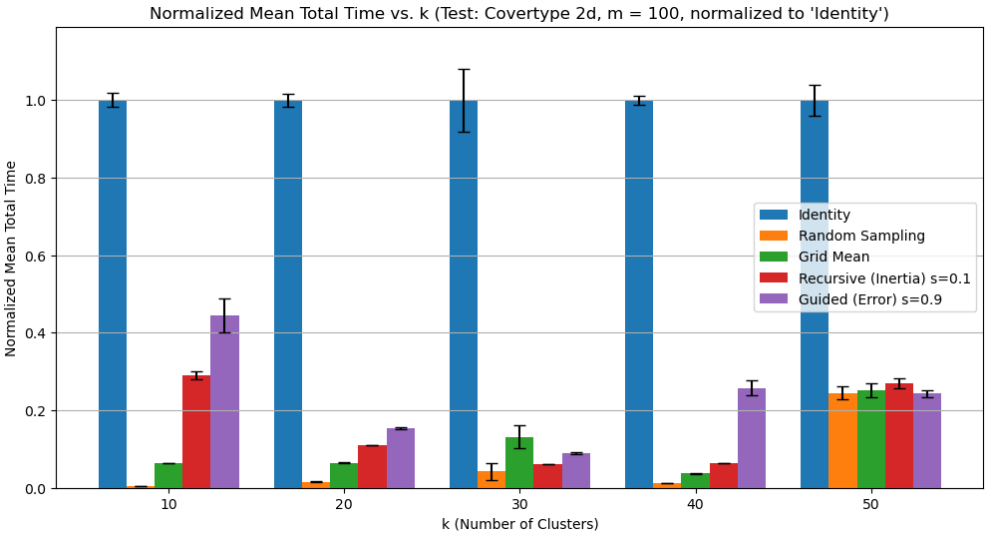


Figure 26: Normalized mean time of all methods vs k (Covertype 2d, m=100)

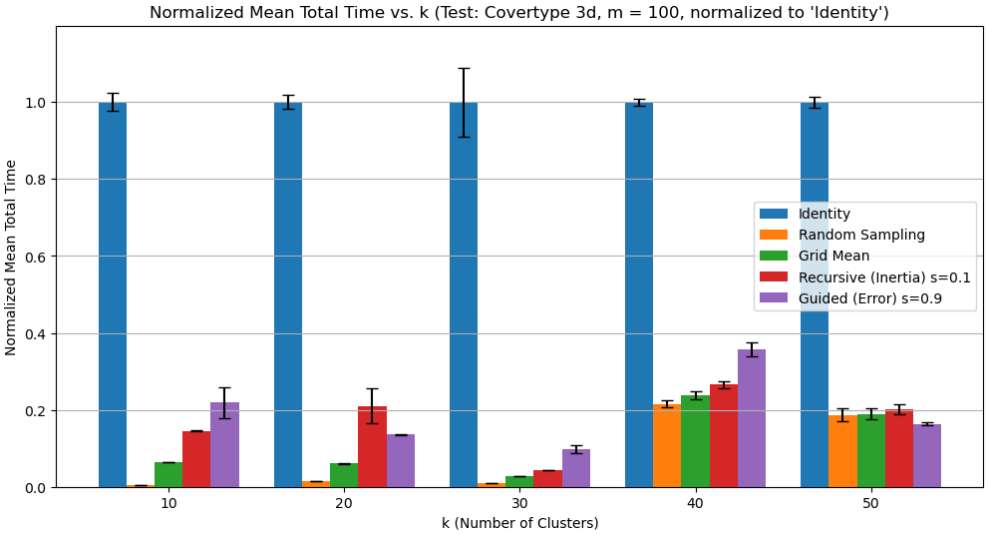


Figure 27: Normalized mean time of all methods vs k (Covertype 3d, m=100)

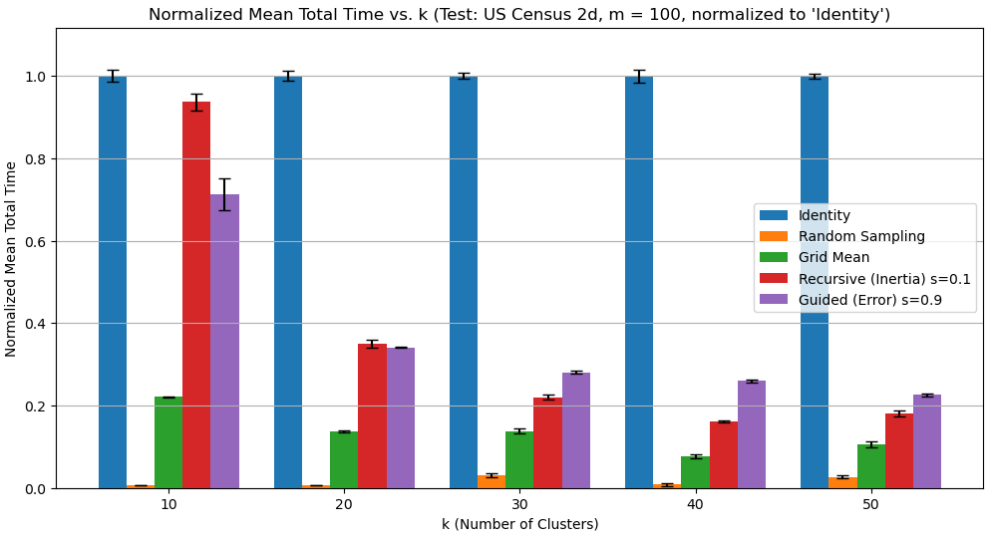


Figure 28: Normalized mean time of all methods vs k (Us Census 2d, m=100)

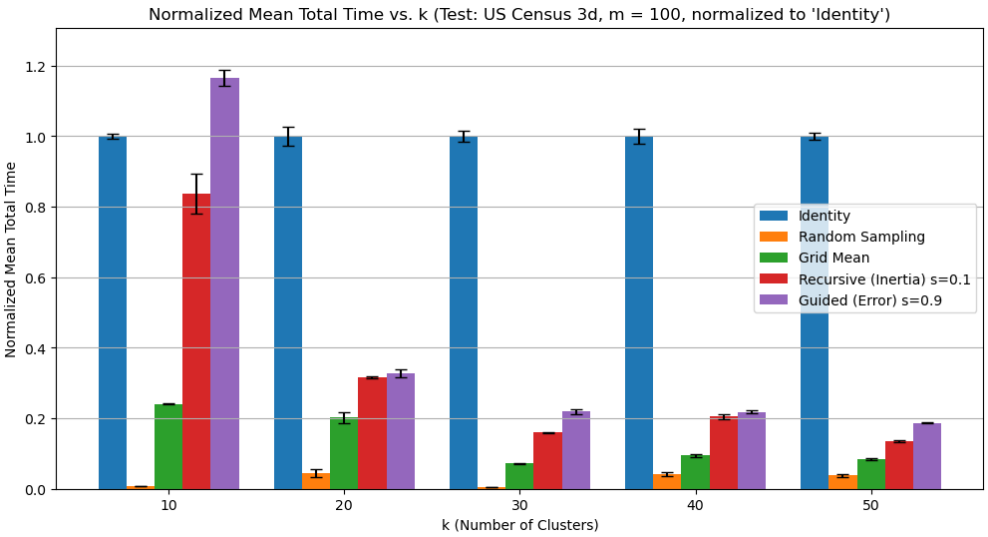


Figure 29: Normalized mean time of all methods vs k (Us Census 3d, m=100)

B Project Plan

Final Year Project Plan: Accelerating K-Means Clustering with Grid-Based Partitioning Coreset Generation

Lucas Marrie

Supervisor: Ran Ben Basat

8 November 2024

1. Background:

K-means clustering is a widely used unsupervised machine learning algorithm that partitions a dataset into k clusters by iteratively assigning points to clusters and updating cluster centroids. Given a set of data points in multi-dimensional space, k-means seeks to minimize variance within clusters, but this process requires repeated distance calculations between points and cluster centroids, which becomes computationally expensive as the size of datasets increases. As data volumes grow, this computational cost poses a significant challenge.

One approach to mitigate these challenges is the use of coresets: small, representative subsets of the dataset that approximate the full data while reducing computational load. By running k-means on a coreset instead of the entire dataset, it is possible to achieve a close approximation of the clustering results while saving substantial computational time. However, generating a coreset itself must be computationally efficient; otherwise, the time savings from clustering on a coreset would be offset by the overhead of coreset generation. Thus, a coreset generation algorithm must be substantially faster than the standard k-means algorithm to be truly advantageous.

2. Aims and Objectives

2.1 Aims

The primary aim of this project is to develop a grid-based partitioning coreset algorithm for k-means clustering that significantly reduces computational time while preserving a good degree of clustering accuracy, specifically targeting low-dimensional data such as 2D and 3D datasets. While some reduction in clustering accuracy is anticipated due to the coreset approximation, a key goal is to minimize this impact and ensure that the algorithm balances speed with acceptable accuracy levels. If time permits, further optimizations will be explored to adapt the algorithm for higher-dimensional data.

2.2 Objectives

- Conduct a review of research surrounding coreset algorithms, with a focus on grid-based methods and applications in low-dimensional data.
- Design a grid-based partitioning coreset generation algorithm suitable for 2d and 3d datasets and develop a sampling strategy that captures the representative structure of each partition. that significantly accelerates k-means clustering.
- Implement a corset generation algorithm that applies the grid-based partitioning and sampling approach, to create a smaller representative approximate of the full data.
- Integrate the coreset into the k-means clustering process and evaluate its accuracy and speed to other existing coreset generation algorithms.

3. Expected Deliverables

3.1 Minimum Deliverables

- Literature review on existing coreset generation algorithms.
- Detailed specifications on grid partitioning, sampling strategy, and algorithm structure for generating coresets in low-dimensional data (2D and 3D).
- An implementation of the grid-based coreset generation algorithm and integration with a k-means clustering module.
- An evaluation of the algorithm's performance in terms of clustering speed and accuracy compared to standard k-means on the full dataset, and comparison against other common coreset techniques.

3.2 Desired Deliverables

- An adapted version of the algorithm with optimizations to handle higher-dimensional datasets effectively
- Add a visualisation component that would allow to visualise the coreset algorithm in action as well as the resulting clustering.
- Add an interface for parameter tuning to allow the algorithm to lean more on either speed or accuracy

4. Work plan

Time Period	Duration	Task	Deadline
1 October – 1 November	4 weeks	Initial review	-
2 November – 8 November	1 week	Project Plan	8 November
9 November – 13 December	5 weeks	Research and Literature review	-
14 December – 6 January	3 weeks	Algorithm Design	-
7 January – 17 January	2 weeks	Initial implementation	-
17 January – 24 January	1 week	Interim Report	24 January
25 January – 9 February	2 weeks	Continued Implementation and initial testing	-
10 February – 2 March	3 weeks	Design and Implement optimizations and extra features	-
3 March – 23 March	3 weeks	Benchmarking against k-means and other coresets	-
24 March – 6 April	2 weeks	Final Testing and Evaluation	-
6 April – 25 April	3 weeks	Finalise Report	25 April

Ethics Review

Since the project solely involves algorithmic work with no ethical implications for privacy, consent, or harm, an ethics review is not necessary.

C Interim Report

Interim Report:
Accelerating K-Means Clustering with Grid-Based Partitioning Coreset Generation

Lucas Marrie

Supervisor: Ran Ben Basat

6 February 2025

Progress made:

I started with making a thorough search of research papers related to grid-based partitioning methods or core-set generation methods for k-means. My search has led me to find a resourceful survey paper comparing a list of relevant coreset methods for k-means, benchmarking them with each other. Additionally, I have found 7 different papers showcasing innovative k-means core-set generation methods, as well as 5 different papers utilizing grid-based partitioning methods. Despite the grid-based methods not directly being related to core-set generation, they mostly relate to clustering methods, which closely relate to k-means, and by extension could be effectively utilized to develop a novel algorithm for core-set generation.

Based off the gird-based methods that were analyzed I brainstormed a prototype algorithm for the project. The basic algorithm begins with splitting the dataset into an infinite grid, where each point's grid position can be found by calculating indices based off its coordinates in N dimensional space. Points in the same grid position are then grouped up and averaged to get the core-set point (with a cumulative weight of all the points). After K-means is performed, the core-set point can then be reversibly mapped.

Extensions to the algorithms I've already devised would be to increase the granularity of the grid of individual grid cells to spread out the data a bit more to preserve better accuracy. This granularity change would be determined by parameters that could either be input or be calculated based on statistical properties like means and variance.

As there are a lot of different possible optimizations that could be made, the priority would be to establish a thorough testing framework to mix and match the best combination of optimizations.

Remaining work:

The first work that needs to be completed is the literature review. I must complete a comprehensive literature review of the papers I've used to demonstrate the background information and related work. The primary focus of the review will be to discuss the papers demonstrating the k-mean coreset generation methods, as well as the papers on grid-based partitioning methods. As the research phase has already been finished, I plan to complete this step by reading week.

Following the completion of the literature review, I need to implement the initial version of the algorithm as detailed in the previous section. Afterwards, I will set up a testing framework to be able to properly evaluate and compare the algorithm to the base k-means. Given that most of the design has been completed I believe completing this step by the end of reading week would be a realistic timeline.

Depending on the remaining time, I will extend the algorithm to include additional improvements/optimizations (as outlined in the previous sections) as well as additional helpful features such as an algorithm visualizer and interface for parameter tuning.

The final practical step would include a thorough test and analysis to be included in the report, and if time allows a comparison to other coreset solutions. Implementations and benchmarking code were provided by a survey paper comparing coresets, however, integrating my solution with the code may be problematic too due to having to adapt it to a different language and libraries. The time I will dedicate to this step and the prior one will depend on the remaining time I have left; I believe at least 2 weeks should be left to finalize the report; therefore these additional changes can be made up until 2 weeks before the deadline.

Afterwards, all that is left would be to write up the final report. This would include the literature review, details of the algorithm, and a comprehensive evaluation of the algorithm's capabilities and performance.